Resource Management for Serverless Edge Computing: A Comprehensive Survey

PRIYANKA SHRESTHA, The University of Georgia, USA
SUSHRUTH HARSHA, The University of Georgia, USA
AVANI PATHAK, The University of Georgia, USA
JIANWEI HAO, Governors State University, USA
IN KEE KIM*, The University of Georgia, USA

Serverless computing offers elastic scaling, event-driven execution, and minimal operational overhead, while edge computing enables low-latency processing closer to data sources. Serverless edge computing (SEC) combines these paradigms for applications such as IoT analytics, video processing, and smart city services. However, edge deployments are heterogeneous and resource-constrained, fundamentally changing the resource-management problem. Despite growing research interest, no existing survey provides a comprehensive resource-management perspective on SEC. This survey addresses this gap by synthesizing over 200 studies published through early 2026. We present a unified taxonomy covering autoscaling, placement, scheduling, offloading, isolation, energy-aware management, network coordination, and state management. We also examine open-source platform adaptations and identify open research challenges. This work provides a structured reference for researchers and practitioners building efficient SEC platforms.

CCS Concepts: • **General and reference** → **Surveys and overviews**; • **Computer systems organization** → **Cloud computing**; *Heterogeneous (hybrid) systems.*

Additional Key Words and Phrases: serverless edge computing, Function-as-a-Service, resource management, task offloading, autoscaling, function placement and scheduling, cold start, multi-tenancy isolation, energy-aware computing

ACM Reference Format:
Priyanka Shrestha, Sushruth Harsha, Avani Pathak, Jianwei Hao, and In Kee Kim. 2026. Resource Management for Serverless Edge Computing: A Comprehensive Survey. *ACM Comput. Surv.* 1, 1 (February 2026), 35 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Serverless computing is increasingly adopted for cloud-native applications [25, 169]. Delivered through Function-as-a-Service (FaaS), it enables developers to deploy stateless, event-triggered functions without managing underlying infrastructure [194]. Major cloud-based serverless platforms, *e.g.*, AWS Lambda, Azure Functions, and Google Cloud Functions, automatically provision containers or microVMs [2, 209], scale resources in response to workload demand,

*Corresponding author.

Authors' Contact Information: Priyanka Shrestha, priyanka.shrestha@uga.edu, The University of Georgia, USA; Sushruth Harsha, sushruth.harsha@uga.edu, The University of Georgia, USA; Avani Pathak, avani.pathak@uga.edu, The University of Georgia, USA; Jianwei Hao, jhao@govst.edu, Governors State University, USA; In Kee Kim, inkee.kim@uga.edu, The University of Georgia, USA.

Manuscript submitted to ACM

Table 1. Summary of surveys related to serverless, edge, and resource management.

	Existing Review Paper	Serverless-Edge Computing (SEC)	Cloud Serverless	Edge Computing	Resource Management
Group #1	[12], [95], [82] [155], [203], [8]	✓	✗	✗	✗
Group #2	[61], [45], [194], [172], [111], [24], [73]	✗	✓	✗	✗
	[184], [60], [125], [127]	✗	✓	✗	✓
Group #3	[3], [191], [121], [131], [75], [185]	✗	✗	✓	✓
Our work		✓	✗	✗	✓

and charge based on actual execution time. Due to these advantages, serverless computing is now widely used across diverse application domains [172], and the global serverless market is expected to surpass $44 billion by 2029 [129].

At the same time, there is an increasing number of applications that demand low latency responses and data locality. Specifically, latency-sensitive use cases, *e.g.*, autonomous vehicles [214], industrial automation [100], and augmented reality [170], impose stringent latency requirements that are difficult to achieve by centralized cloud architectures due to propagation/network delays [167, 175]. In addition, increasing concerns about data privacy, bandwidth costs, and regulatory compliance drive the need to process data near its source [64, 88, 121]. These requirements have led to the adoption of edge computing, which distributes storage and computation across gateways, micro data centers, and end devices to support near-source data processing [98, 121, 168].

Serverless Edge Computing (SEC) synthesizes the serverless computing model with the physical proximity of edge infrastructure: it combines the fine-grained elasticity and operational simplicity of FaaS with the low latency and data locality of edge computing. SEC enables a range of applications, including real-time IoT analytics [64, 81], interactive AR/VR [11], smart manufacturing [57], and latency-sensitive AI inference at the edge [207]. By deploying serverless functions closer to end users and data sources, SEC supports low-latency, lightweight, event-driven computing while mitigating the performance bottlenecks of centralized clouds.

However, deploying serverless computing at the edge introduces new challenges for resource management. Edge nodes and devices are inherently resource-constrained, heterogeneous, and distributed. They often operate with limited energy budgets and may experience intermittent connectivity [81, 88, 121]. Design assumptions used when developing cloud-based serverless systems, *e.g.*, infinite capacity for autoscaling and stable networking for state management, can break down in edge environments. For example, cold-start latencies are amplified by slower edge processors, orchestration overhead consumes scarce device energy, and static provisioning leads to poor device utilization. Therefore, resource management becomes a key design consideration for SEC performance, cost efficiency, and reliability.

Existing surveys have examined SEC architectures [8, 12, 82, 95, 155, 203], serverless computing [24, 45, 61, 73, 111, 172, 194] or its resource management in cloud settings [60, 125, 127, 184], and edge/fog resource management [3, 75, 121, 131, 185, 191], largely in isolation. However, no existing work centers on resource management within the SEC as a joint problem, where serverless control loops interact tightly with edge heterogeneity and limited connectivity. Table 1 positions our survey relative to these related works and highlights the dimensions we uniquely cover.

This survey aims to address this gap by focusing on the resource management problem in SEC, specifically how platforms allocate, place, and schedule ephemeral function executions across heterogeneous, distributed, and resource-constrained edge infrastructure. We synthesize state-of-the-art research that improves SEC performance and efficiency under these constraints and organize the literature around the operational decisions that SEC platforms must make at runtime.

To guide this survey and make our synthesis explicit, we structure the review around the following research questions (RQs):

- **RQ #1:** What *system constraints* distinguish SEC from cloud serverless, and how do they impact resource management?
- **RQ #2:** What are the *core decision dimensions* for SEC resource management, and what control knobs characterize each?
- **RQ #3:** Which *resource management approaches* are applied across these dimensions, what objectives do they optimize, and what are their limitations?
- **RQ #4:** What *evaluation gaps* and *open research problems* remain for reproducible SEC resource management research?

In answering these RQs, our work goes beyond a simple enumeration of techniques; we provide a critical synthesis that connects isolated research efforts to the fundamental constraints of the edge. Specifically, this work makes the following research contributions:

- **A characterization of SEC resource management challenges (RQ #1; §2.2, §2.3.1).** We analyze how edge constraints, which include hardware heterogeneity, resource scarcity, spatial distribution, and energy variability, invalidate cloud-centric assumptions. This yields a clear problem framing across the function lifecycle (invocation, provisioning, execution, teardown) and deployment tiers (device–edge–cloud).
- **A unified taxonomy of SEC resource-management mechanisms (RQ #2; §2.3.2).** We organize existing SEC research by decision dimension, *e.g.*, autoscaling, placement, scheduling, offloading, isolation, energy/network awareness, and state management. Each dimension is defined by its control knobs, observable signals, and optimization objectives.
- **A comparative analysis of resource management approaches (RQ #3; §3.1 – §3.6).** We survey existing approaches within each dimension, mapping them to optimization goals, system assumptions, and evaluation settings. For each category, we identify strengths, limitations, and gaps that remain unaddressed.
- **An analysis of evaluation gaps and future directions (RQ #4; §3.7, §5).** We identify cross-cutting open problems including holistic multi-objective control, accelerator-aware orchestrations, and stateful execution. We also evaluate current benchmarking practices and identify requirements for reproducible SEC evaluation.

The remainder of the paper is organized as follows. §2 introduces background on SEC and the key constraints that shape resource management. §3 surveys resource management approaches in SEC. §4 surveys how open-source serverless platforms are adapted for edge resource management. §5 discusses the future research directions and open research problems. §6 concludes the paper.

2 Background

This section provides background for understanding SEC resource management. We first introduce the background of serverless computing, edge computing, and SEC (§2.1). We then describe the edge hardware landscape (§2.2), identify resource management challenges and mechanisms (§2.3), and position this survey relative to prior work (§2.4).

2.1 Serverless, Edge, and SEC

SEC combines serverless and edge computing [8, 155]. Serverless computing provides an event-driven execution model centered on short-lived function instances, while edge computing brings compute and storage closer to data sources to reduce latency and bandwidth usage. SEC inherits serverless elasticity and programming simplicity, but must operate at

the edge, where resources are limited, hardware is heterogeneous, and connectivity is variable. We introduce serverless computing (§2.1.1), edge computing (§2.1.2), and their convergence in SEC (§2.1.3).

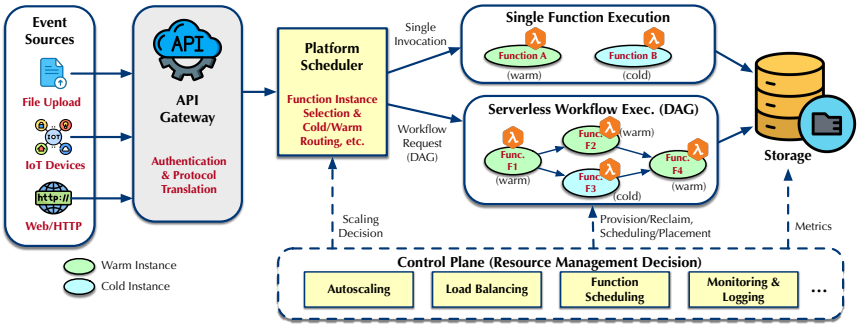


Fig. 1. Serverless execution pipeline: from event sources through scheduling to function instances.

2.1.1 Serverless computing. Serverless computing is an execution model in which developers deploy individual functions that the platform provisions, scales, and reclaims on demand, eliminating explicit infrastructure management [194]. It is typically delivered through Function-as-a-Service (FaaS), where applications are decomposed into fine-grained, event-driven, stateless functions. Figure 1 illustrates the serverless execution pipeline. Incoming requests enter through an API gateway, which handles authentication and protocol translation. The platform scheduler then maps each invocation to a function instance (*e.g.*, container or microVM). If a warm instance is available, the request is served immediately; otherwise, the platform provisions a new runtime, introducing cold-start latency. Cold-start includes runtime initialization, code loading, memory allocation, and often container image retrieval [45, 61]. Function instances are reclaimed after an idle period, freeing resources but increasing cold-start frequency [173]. As shown in Figure 1, the underlying control plane continuously monitors function instances and drives autoscaling, load balancing, and scheduling decisions based on runtime metrics.

Beyond single-function invocations, many real-world applications require multi-function coordination via workflow engines (*e.g.*, AWS Step Functions and Azure Durable Functions), where functions form a DAG (Directed Acyclic Graph) with dependencies that the scheduler must jointly manage (see §3.2).

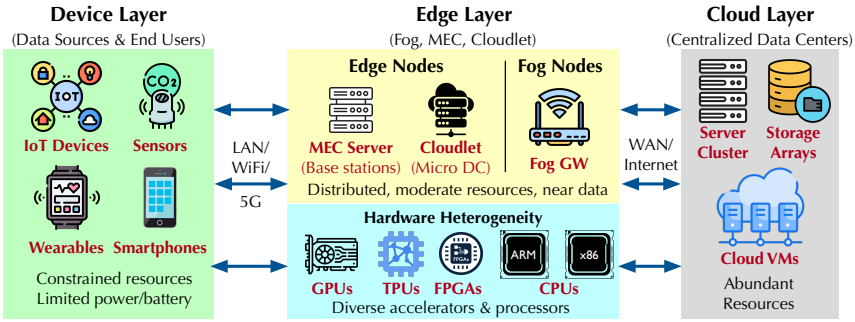


Fig. 2. The device–edge–cloud continuum with three deployment tiers.

2.1.2 Edge computing. Edge computing deploys computation and storage closer to data sources and end users rather than relying solely on centralized cloud data centers [98, 121, 168]. By placing resources near the data, edge computing

reduces end-to-end latency and data transfer to the cloud, while the cloud remains essential for large-scale analytics and workloads that require substantial computational resources. Edge computing environments introduce several constraints that directly affect SEC resource management. For example, edge nodes (or devices) are distributed across dispersed sites such as gateways, base stations, and micro data centers. Hardware heterogeneity, including diverse accelerators (§2.2), complicates placement and scheduling, and limited compute, memory, and energy budgets further constrain resource management [99, 199], as discussed in §2.3. Several deployment models exist within the edge paradigm, including fog computing [63], multi-access edge computing (MEC) near cellular base stations [166] and cloudlets that provide micro data centers one hop from mobile devices [167, 168]. Figure 2 illustrates the device–edge–cloud continuum spanning these models.

2.1.3 Serverless Edge Computing. SEC extends the serverless execution model to edge environments. Instead of executing functions exclusively in cloud data centers [31], SEC runs functions on edge clusters or edge devices (*e.g.*, Raspberry Pi, NVIDIA Jetson, gateways, and micro data centers). Note that CDN-based platforms such as Cloudflare Workers[1] and AWS Lambda@Edge[2] execute functions at network edge points but operate on provider-managed infrastructure with abundant resources and always-on connectivity; we consider these outside the scope of SEC (see §4). This brings execution closer to data sources and users, reducing response time and bandwidth usage while preserving the key advantages of serverless models, including event-driven execution and fine-grained elasticity.

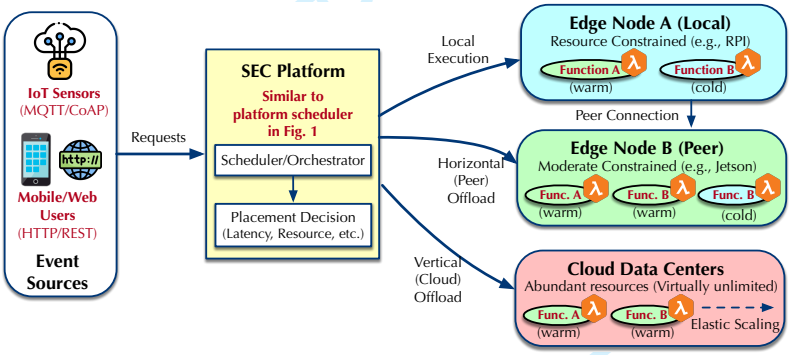


Fig. 3. SEC execution model: function invocations are routed (or offloaded) to local edge, peer edge, or cloud based on runtime conditions.

Figure 3 shows the SEC execution model. When a function is invoked, the SEC platform decides whether to execute it locally on the receiving edge node, offload it horizontally to a peer edge node with available resources, or redirect it vertically to the cloud. These decisions depend on runtime conditions including resource availability, warm instance status, and network latency.

SEC introduces a resource management challenge not present in cloud serverless. For example, SEC platforms must decide whether to execute functions locally at the edge or offload them to the cloud, balancing latency, resource availability, and cost, under constraints such as device/accelerator heterogeneity, cold-start latency, and network variability (§3). These challenges are particularly difficult to address in SEC's target application domains, such as IoT analytics, smart cities, industrial automation, healthcare monitoring, AR/VR, mobile gaming, and edge ML inference,

[1]https://workers.cloudflare.com/
[2]https://aws.amazon.com/lambda/edge/

where stringent latency requirements and geographic distribution make locality-aware function placement and rapid scaling essential for meeting performance objectives [135, 160, 198].

2.2 Edge Hardware and Heterogeneity

Unlike the cloud, where relatively homogeneous servers are found in data centers, the edge infrastructure is characterized by inherent hardware heterogeneity. Edge nodes/devices span a wide spectrum of compute capabilities, memory footprints, and power budgets. This heterogeneity leads to significant performance variability, making a function's execution time and resource footprint strictly host-dependent. Such variability complicates the efficient management of edge resources and serverless applications, demanding resource-aware orchestration mechanisms [95].

Edge device classes. Edge devices range from ultra-low-power microcontrollers (MCUs) to compact x86 edge servers. MCUs (*e.g.*, ESP32, STM32) target minimal power consumption and increasingly support lightweight AI pipelines via TinyML [84]. Mid-tier devices, such as Raspberry Pi-class single-board computers (SBCs), offer modest compute and memory for general-purpose tasks but are susceptible to performance degradation and resource contention when hosting concurrent serverless functions. At the upper tier, compact x86 systems (*e.g.*, Intel NUC) provide desktop-grade throughput, making them suitable for heavier analytics and orchestration roles, although their deployment is often sparse compared to the dense population of sensor-level nodes.

AI accelerators at the edge. Modern SEC increasingly targets AI-driven workloads, making specialized accelerators central to the hardware landscape. Edge GPUs (*e.g.*, NVIDIA Jetson family) improve inference throughput through parallelism, whereas edge TPUs (Tensor Processing Units) like Google Coral and NPUs (Neural Processing Units) provide high operations per second (TOPS) with superior energy efficiency for constrained deployments [72]. FPGAs (Field-Programmable Gate Arrays) and ASICs (Application-Specific Integrated Circuits) offer deterministic latency for specific models and sensing pipelines, but they often require exclusive resource access and pose challenges for dynamic abstraction [42]. For SEC platforms, these specialized resources demand placement and scheduling policies that explicitly model accelerator availability, architectural compatibility, and contention to avoid resource mismatches.

2.3 Challenges and Mechanisms in SEC Resource Management

SEC platforms face unique resource management challenges. They must provide serverless elasticity (*e.g.*, rapid scaling and automatic provisioning) on edge infrastructure that is resource-constrained, heterogeneous, and distributed. Unlike cloud serverless platforms with virtually unlimited resources, SEC must continuously adapt placement, scaling, and scheduling decisions on its unique computing infrastructure. This section first identifies ten key challenges (§2.3.1) and then defines the resource management mechanisms that address them (§2.3.2).

2.3.1 Key challenges. The challenges presented below emerge from analyzing studies included in this survey. We organize them into four categories: (1) *infrastructure-level challenges* that stem from edge hardware and platform limitations, (2) *runtime-level challenges* related to function lifecycle and scheduling, (3) *application-level challenges* involving state and workflow complexity, and (4) *system-wide challenges* related to spatial distribution and evaluation methodology. These challenges directly motivate the taxonomy of resource management techniques surveyed in §3.

Infrastructure-level challenges. These challenges arise from the fundamental differences between cloud data centers and edge infrastructure.

(i) **Cloud-oriented serverless platforms are not edge-optimized.** Traditional serverless designs assume abundant resources and stable network connectivity, which do not hold at the edge. Cloud-optimized FaaS

stacks can be too heavyweight for constrained edge nodes [146, 148], centralized scheduling/orchestration introduce delay in distributed edge deployments [35], and the performance behavior of serverless functions on heterogeneous edge hardware is not yet extensively characterized [147].

(ii) **Hardware heterogeneity poses significant challenges for function placement and scheduling decisions.** Edge deployments span heterogeneous CPU architectures (ARM/x86), runtime stacks, and accelerators, and low-powered nodes can overload easily under fluctuating ingestion rates [6, 161]. Orchestrating functions across diverse devices can make QoS compliance difficult [96, 165], and current SEC platforms often lack fine-grained accelerator support for GPU sharing and contention management [160]. These factors require placement and scheduling algorithms that explicitly model hardware heterogeneity.

(iii) **Current SEC platforms lack energy-aware resource management.** Edge nodes often have limited power budgets, and some edge deployments rely on batteries or renewable energy, making energy availability variable. Although energy-aware approaches have been proposed in the literature [5, 20, 112, 142], current platforms have not yet adopted them.

Runtime-level challenges. These challenges concern the dynamic behavior of serverless functions during execution.

(iv) **Bursty workloads challenge scaling under resource constraints.** Limited resources on edge nodes make handling sudden bursts challenging, requiring scaling mechanisms specifically designed for resource-constrained SEC environments. Reactive scaling mechanisms often cannot respond fast enough, leading to increased latency. For example, Kubernetes HPA-style scaling can lag burst traffic [187], and threshold-based policies can be brittle because optimal utilization targets vary across functions and are difficult to configure [110].

(v) **Balancing latency, resource usage, and offloading cost is challenging.** SEC platforms must decide whether to execute functions locally at the edge or offload them to the cloud. Local execution reduces latency but consumes scarce edge resources; offloading frees local resources but incurs network delay and data transfer costs. These trade-offs are difficult to optimize because node performance varies and workload characteristics are often unpredictable. AI workloads amplify these tensions because model sizes and memory footprints can be substantial. For example, deploying large DNN (Deep Neural Network) models on resource-constrained edge nodes can increase processing time and memory pressure [66].

(vi) **Cold-start latency is amplified at the edge, and mitigation requires multi-objective optimization.** While cold-start affects all serverless platforms, edge environments significantly amplify the problem as networks are unreliable, nodes are constrained, and image registries may be remote. Measurements show that cold-start delay can dominate execution time for certain workloads [70, 200]. Common mitigations such as container caching, pre-warming, and lightweight isolation can reduce cold-start latency but may consume additional memory and power/energy [1, 28, 134, 140], motivating adaptive approaches surveyed in §3.1 and §3.4.

Application-level challenges. These challenges arise from SEC application requirements.

(vii) **Stateful execution is difficult on distributed, constrained edge deployments.** While serverless was originally designed for stateless functions, many edge applications require state across invocations [74]. Managing state across distributed edge nodes requires balancing locality, consistency, and migration costs [34, 37]. The absence of shared storage at the edge complicates state placement and replication [74]. Some approaches address state locality and access latency [195], but comprehensive solutions are lacking.

Table 2. Relationship between key challenges (§2.3.1), resource management mechanisms (§2.3.2), and corresponding survey sections. *Note the challenge (i) - 'not edge optimized' is jointly addressed by all mechanisms.*

Resource Mgmt. Mechanisms	SEC Challenges
Autoscaling and load balancing (§3.1)	(iv) Bursty workloads, (vi) Cold-start
Placement and scheduling (§3.2)	(ii) Heterogeneity, (viii) Workflow orch., (ix) Spatial dist.
Dynamic function offloading (§3.3)	(v) Offloading trade-offs
Multi-tenancy and isolation (§3.4)	(vi) Cold-start, (ii) Heterogeneity
Energy-aware management (§3.5)	(iii) Energy constraints
State management (§3.6)	(vii) Stateful execution
Benchmark and evaluation tools(§3.7)	(x) Lack of standard benchmark

(viii) **Workflow orchestration and long-running tasks complicate scheduling.** Edge workloads often involve multi-step pipelines and continuous processing, where frequent container provisioning and termination increase scheduling overhead [89]. Supporting long-running or stateful steps is challenging on resource-constrained edge nodes, motivating checkpointing, migration, and QoS-aware orchestration [164]. Workflow scheduling must jointly manage dependencies, resource constraints, and latency targets [146, 201, 202].

System-wide challenges. These challenges span the entire SEC ecosystem.

(ix) **Spatial distribution, mobility, and network variability require distributed coordination.** SEC deployments span spatially distributed edge nodes. Centralized schedulers can become bottlenecks, introducing network latency and reducing responsiveness, which motivates decentralized scheduling and migration techniques [68, 83, 162]. User mobility and fluctuating demand can create spatial hotspots and resource imbalance, further complicating placement and scaling.

(x) **SEC lacks standardized benchmarks and evaluation methodologies.** Without common benchmarks, reproducible evaluation and comparison across heterogeneous edge hardware is difficult. Studies show that devices such as Raspberry Pi and Jetson Nano exhibit markedly different cold-start and warm-start times and concurrency behavior [159], yet there is no standard way to measure and compare these differences. This lack of standardization hinders progress in understanding and optimizing SEC resource management.

2.3.2 Resource Management Mechanisms. SEC platforms address these challenges through the mechanisms summarized in Table 2. §3 surveys how the literature implements each mechanism. Below, we briefly define each mechanism to establish terminology for the remainder of this survey.

Autoscaling and load balancing. Autoscaling determines the number of function instances and their resource capacity under changing demand, balancing latency against resource consumption. Common approaches include workload-based scaling (*e.g.*, request-driven) and resource-based scaling (*e.g.*, CPU utilization thresholds). Scaling can be reactive or predictive [149], and the edge setting makes robust burst handling and heterogeneity awareness particularly important [16, 49]. Load balancing routes function invocations across edge nodes, prioritizing warm instances (to avoid cold-starts), and considering node capacity and network conditions. Centralized load balancers can become bottlenecks in distributed edges, motivating distributed designs that minimize latency.

Placement and scheduling. Placement selects the execution node for each function, matching resource requirements to node capacity. Scheduling decides execution order and resource sharing within a node. Decisions consider resource availability, warm instance availability, deadlines, network conditions, and data locality to meet application requirements.

Dynamic function offloading. Offloading decides at runtime whether to execute a function locally or redirect it to peer edge nodes or the cloud based on current workload and network conditions, trading off latency, resource availability, and data transfer cost.

Multi-tenancy and isolation. SEC platforms run functions from multiple tenants on shared edge hardware, requiring isolation to prevent interference. Common mechanisms include containers, WebAssembly, and unikernels, each with different security and performance trade-offs. Stronger isolation improves security but increases startup overhead (*e.g.*, cold-start latency [148]).

Energy-aware management. Energy-aware schedulers consider battery state and renewable energy availability when making placement and offloading decisions, sustaining node availability while minimizing energy consumption and environmental impact [5, 20].

State management. State management determines where and how the state is stored, accessed, and migrated so that distributed functions can share and persist information efficiently. Co-locating state with compute can reduce read/write latency [74, 195], but must be balanced against consistency and fault tolerance.

Benchmarking, simulation, and testbeds. Benchmarks are used to quantify function execution latency, cold-start and warm-start behavior, and resource utilization across heterogeneous edge devices [159]. Simulators and emulators can help explore design alternatives such as scheduling policies and scaling strategies before deployment. Testbeds provide controlled environments for validating SEC platforms on real hardware [59, 157].

These interdependencies among mechanisms explain why resource management has become a central research focus in SEC, as reflected in the studies surveyed in §3.

2.4 Related Surveys

Prior surveys on SEC primarily emphasize architectures, protocols, and general feasibility, while surveys on serverless or edge resource management often treat the two paradigms separately. Table 1 (in §1) summarizes existing surveys across three groups and positions our work relative to them.

A first group of surveys focuses on SEC itself, examining architectures, platforms, and feasibility [8, 12, 82, 95, 155, 203]. These include architectural taxonomies [12], network-centric deployment analyses [203], and systematic mappings of IoT serverless computing [95]. However, these works do not center on resource management as their primary organizing principle.

A second group surveys serverless computing more broadly, including cold-start analysis, autoscaling strategies, and general research directions [24, 45, 61, 73, 111, 172, 194]. Holistic views of resource management are synthesized in [60, 125, 127]. Although some of these works address resource management, they focus on cloud environments rather than edge-specific constraints.

A third group surveys edge/fog resource management, often incorporating AI-based techniques [3, 75, 121, 131, 191]. These works provide valuable methodological context but do not address serverless-specific concerns such as cold-starts, scale-to-zero, and multi-tenancy isolation.

Our positioning. To the best of our knowledge, no existing survey centers resource management within SEC as a joint problem, where serverless control loops and edge constraints are treated together. As shown in Table 1, our survey uniquely combines SEC focus with resource management by (i) defining SEC-specific objectives and constraints, (ii) presenting a taxonomy across scaling, placement, isolation, state, and energy/network awareness, and (iii) synthesizing evaluation methodologies and open research directions grounded in SEC realities.

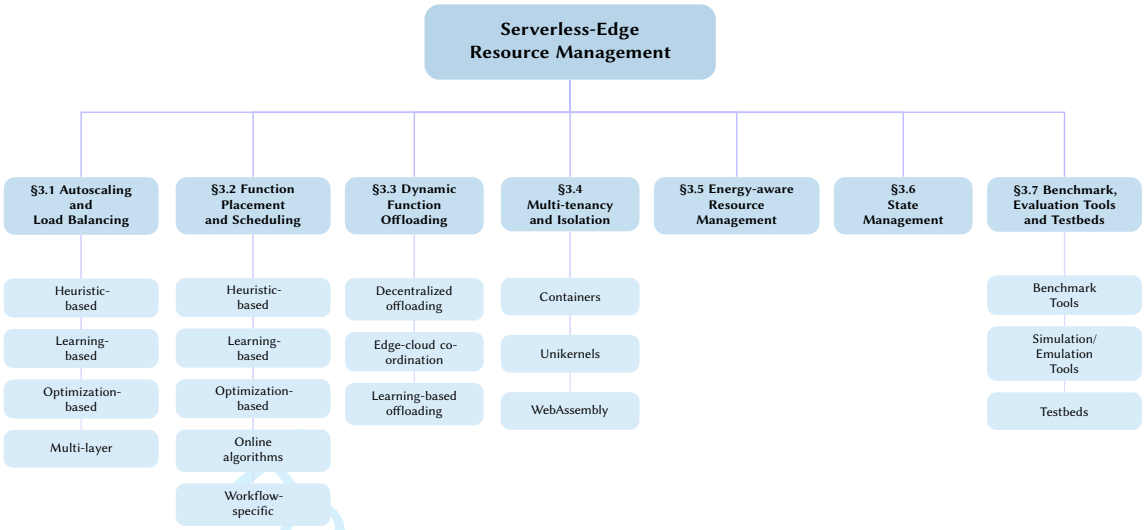


Fig. 4. Taxonomy of Resource Management in Serverless Edge Computing.

3 Current Resource Management Approaches

This section surveys representative resource management approaches developed for SEC. We organize the literature into eight subsections: (1) *autoscaling and load balancing* (§3.1), (2) *function placement and scheduling* (§3.2), (3) *dynamic function offloading* (§3.3), (4) *multi-tenancy and isolation* (§3.4), (5) *energy-aware management* (§3.5), (6) *state management* (§3.6), and (7) *evaluation methodologies* (§3.7). Figure 4 illustrates this taxonomy.

3.1 Autoscaling and Load Balancing

Autoscaling in SEC aims to meet application performance targets (*e.g.*, response time, throughput) under dynamic demands without over-consuming constrained edge resources. Unlike cloud serverless platforms that can provision function instances with relatively predictable startup times, SEC platforms often face amplified cold-start latencies due to slower edge processors, limited container caching, and constrained memory for keeping warm instances. Because provisioning new replicas takes longer at the edge, reactive scaling mechanisms face extended provisioning delays, motivating proactive strategies that predict future demand or continuously tune scaling thresholds/parameters [16, 49]. Moreover, SEC applications increasingly involve multi-function workflows rather than isolated invocations. Scaling a single function *may* not improve end-to-end latency if other functions in the workflow remain bottlenecks, requiring autoscaling that incorporates cross-function dependencies, such as inter-stage queueing delays and cascading cold-starts.

Load balancing in SEC is complicated by hardware heterogeneity of edge devices/nodes, which differ in compute capability, memory capacity, and accelerator availability. Therefore, load balancing decisions for SEC cannot simply treat nodes as interchangeable, which differs from cloud environments where backends are typically homogeneous.

3.1.1 Current Approaches. We organize existing approaches into four categories based on their underlying control principles: heuristic-based, ML-based, optimization-based, and multi-layer approaches. Table 3 summarizes the strengths and limitations of these approaches.

Table 3. Comparison of Autoscaling and Load Balancing Approaches.

Approach	Strengths	Limitations
Heuristic-based (threshold/rule) [6, 41, 56, 133, 150]	Lightweight, low overhead, interpretable	Reactive, struggles with bursty workloads
Learning-based (learned policies) [15, 16, 18, 33, 48, 49, 51, 69, 186]	Adaptive, handles non-stationary demand	Requires training data, less interpretable
Optimization-based (model-driven) [19, 102, 156, 187]	Principled, optimizes explicit objectives	Requires accurate models, high computational cost
Multi-layer (cross-tier coordination) [40, 143]	Handles edge-cloud hierarchy	Increased complexity, coordination overhead

Heuristic-based. Heuristic-based approaches trigger scaling and load balancing decisions when observed resource metrics cross predefined thresholds, offering low computational overhead. Pranata *et al.* [150] analyze how different CPU utilization thresholds affect response time and resource consumption, providing guidance for threshold selection. Dantas *et al.* [41] use utilization thresholds to redirect requests between edge and cloud when local resources are constrained. However, as fixed thresholds may not adapt well to changing workload patterns, Gand *et al.* [56] propose a fuzzy-logic controller that adapts scaling decisions based on real-time conditions.

Beyond single-metric thresholds, Aslanpour *et al.* [6] propose a multi-objective approach that balances throughput, energy efficiency, and inference precision according to policy priorities. Queue-aware heuristics offer finer-grained signals than resource utilization alone. Mu [133] uses queue-level metrics and lightweight workload prediction rather than CPU utilization to guide both autoscaling and load balancing decisions, reducing SLO (Service Level Objective) violations under varying load.

Learning-based. Learning-based approaches learn scaling policies from observed demand patterns, eliminating the need for manually tuned thresholds in heuristic methods [33]. Reinforcement learning (RL) is particularly attractive because it can optimize objectives such as latency and utilization over time while adapting to workload fluctuations. For example, Benedetti *et al.* [16] use Q-learning to tune autoscaler thresholds, keeping utilization high while meeting latency targets via a latency-aware reward function. Filinis *et al.* [51] similarly apply RL with an SLO-oriented reward to balance latency compliance with resource efficiency. Hadjou *et al.* [69] apply RL to heterogeneous edge clusters, adapting scaling decisions based on real-time concurrency and latency.

Deep RL methods extend these ideas to handle more complex scenarios. Tran *et al.* [186] combine deep RL with traffic prediction for resource-quota-based systems, explicitly avoiding actions that exceed quota while reducing SLO violations. Proximal Policy Optimization (PPO) has been applied to Kubernetes HPA (Horizontal Pod Autoscaler) to dynamically configure optimal threshold values [48] and to improve robustness under dynamically fluctuating traffic patterns through state space reduction [49]. To improve portability, some approaches use workload-independent state and action spaces that generalize across heterogeneous pipelines [15, 18].

Optimization-based. Optimization-based approaches formulate serverless autoscaling as constrained optimization problems, explicitly modeling resource limits and latency targets to minimize cost or resource usage subject to performance constraints.

HeROcache [102] targets fine-grained resource allocation in constrained edge settings. It uses greedy optimization with storage-aware heuristics to make per-request scheduling decisions, prioritizing edge nodes where required data is already cached. Tran *et al.* [187] combine workload prediction with resource optimization. Their approach adjusts

both resource allocation and concurrency limits to minimize total resource usage while meeting latency targets. For model-driven control, Bensalem *et al.* [19] model scaling as a Semi-Markov Decision Process that predicts system state changes (*e.g.*, demand spike, queue length) to proactively determine optimal scale-out and scale-in decisions, avoiding the reactive delays of threshold-based approaches. SimuScale [156] uses co-simulation to evaluate different autoscaler parameter configurations (*e.g.*, HPA thresholds) during runtime. By simulating how each configuration would perform under current workload conditions, it selects parameters that reduce SLO violations and resource usage.

Multi-layer Approaches. Multi-layer approaches coordinate autoscaling and load balancing across multiple tiers, such as edge-cloud and IoT-edge-fog deployments. These policies determine how much to scale locally and when to offload bursts to other tiers under constraints like network bandwidth and resource availability. HiveMind [143] adopts a centralized approach where a single controller coordinates scaling decisions across edge and cloud tiers to ensure predictable performance. In contrast, Cicconetti *et al.* [40] develop a dual-layer strategy that combines centralized orchestration for global load optimization with distributed dispatchers for rapid local adaptation.

3.2 Function Placement and Scheduling

Function placement and scheduling determine where a function executes, and which function instance handles an invocation. In SEC, these decisions involve trade-offs between locality, resource contention, compute efficiency, and transmission overhead, further complicated by cold-start latency. Moreover, when applications consist of multiple functions, placement becomes more complex, as dependent tasks must be placed to minimize inter-function communication delays. These challenges have led to a wide range of approaches, from lightweight heuristics suitable for resource-constrained nodes to learning-based methods that adapt to dynamic conditions.

3.2.1 Current approaches. We organize existing approaches into five categories: heuristic-based, learning-based, optimization-based, online algorithms, and workflow-specific approaches. Table 4 summarizes the strengths and limitations of these approaches.

Table 4. Comparison of Function Placement and Scheduling Approaches.

Approach	Strengths	Limitations	Surveyed Work
Heuristic-based (deadline and execution, cold-start, QoS/ SLO, locality and resource, workflow and meta-heuristic)	Lightweight, low overhead, interpretable; well-suited for resource-constrained edge deployments	Typically targets specific objectives; may require reconfig. as workload patterns change	[33, 109, 113, 118, 122, 182] [57, 114, 152, 153, 162, 171] [43, 93, 124, 138, 160, 215] [47, 58, 68, 108, 137, 201]
Learning-based (federated RL, deep RL)	Adaptive under dynamic and partially observable conditions; balances multi-objectives w/o individually tuned rules	Requires training data; prone to overfitting; scalability challenges as state/action space grows	[79, 91, 107, 165, 211, 212] [26, 83, 103, 106, 206, 213]
Optimization-based (ILP, multi-objective, joint optimization, batching)	Principled constraint handling with formal guarantees	Computationally intensive for large-scale deploy; depends on system models	[4, 9, 21, 65, 76, 210] [77, 144, 151, 177, 217] [87, 145]
Online algorithms (instance retention, joint caching/ routing, online learning)	Decisions as requests arrive; well-suited for unknown future demand	May miss global optimality without future visibility;	[28, 97, 140, 198, 200, 216] [27, 174, 188, 204]
Workflow-specific (DAG modeling, dynamic optimization)	Dependency-aware; Handles more complex serverless applications	Coord. complexity grows with DAG depth; most assume static DAG struct.	[43, 116, 120, 136, 202]

Heuristic-based approaches. Heuristic methods are commonly used in SEC function placement due to their low overhead and interpretability, which make them well-suited for resource-constrained edge deployments. We organize heuristic methods into five sub-categories: deadline and execution-aware, cold-start-aware, QoS and SLO-aware, locality and resource-aware, and workflow and meta-heuristic approaches.

(i) *Deadline and execution-aware heuristics* place functions based on timing constraints and runtime trade-offs. SledgeScale [122] dispatches functions to workers based on task deadlines, leveraging lightweight WebAssembly sandboxes that can be instantiated on demand to meet latency targets. EDF-based approaches [182] are designed to meet strict deadline compliance for deadline-constrained applications. Priority-based heuristics [113] jointly consider compute and bandwidth constraints to minimize end-to-end latency. At runtime, ESFF [118] creates new instances only when the expected response time improvement outweighs the cold-start overhead, while AIHO [109] dynamically selects between local execution and multi-hop offloading by balancing resource availability against latency penalties.

(ii) *Cold-start-aware heuristics* exploit pre-initialized instances to reduce cold-start delays. Choupani *et al.* [33] keep warm instances active and prioritize the most recently used instances for incoming requests. funcX [114] utilizes a hierarchical, warming-aware algorithm that prioritizes warm container reuse through randomized load balancing, achieving high scalability with minimal scheduling overhead. Warm-up and routing heuristics [171] explore different request-server matching trade-offs, including greedy, conservative, and balanced strategies.

(iii) *QoS and SLO-aware heuristics* consider multiple criteria (*e.g.*, latency, jitter, cost, and resource constraints) when making placement decisions. HyperDrive [152] employs an SLO-aware multi-criteria scoring framework that first filters nodes capable of meeting SLO requirements, then ranks candidates across the edge-cloud continuum. TEMPOS [57] monitors function execution characteristics and node resource conditions at runtime to determine placement that meets stringent SLO targets for latency and jitter. RightFusion [153] reduces inter-function communication overhead by merging co-dependent functions and placing them together on heterogeneous resources to meet QoS targets. Other approaches address QoS-aware offloading under budget constraints [162] and mobility-driven function migration for smart city services [124].

(iv) *Locality and resource-aware heuristics* consider data location, network proximity, and node capabilities when making placement decisions. Skippy [160] places data-intensive functions near high-speed data stores using a bandwidth graph and storage index, reducing cross-network transfer latency. Declarative languages (*e.g.*, TAPP [138]) allow developers to impose topology constraints and restrict function execution to specific edge locations. On the resource side, NEMO [215] harvests underutilized CPU cycles from mobile edge infrastructure, using historical data to pre-warm frequently invoked functions. ShmFaaS [93] shares DNN models across instances via shared memory, increasing deployment density on resource-constrained nodes.

(v) *Workflow and meta-heuristic approaches* handle multi-function placement where task dependencies and multi-objective trade-offs expand the search space, making exact optimization computationally prohibitive. Workflow-aware methods [68] prioritize warm container reuse with bid-based scheduling to reduce workflow execution time while maintaining fairness across concurrent workloads. HEFTLess [47] employs a bi-objective ranking heuristic to jointly minimize makespan and cost across the edge-cloud continuum. PLAYS [58] models DNN inference as a DAG and places functions across edge nodes to minimize queueing delays, achieving 28% – 40% latency reduction. Meta-heuristics such as PSO-GA hybrids [108, 201] and ant colony optimization [137] search for near-optimal placements balancing latency, energy, and cost across edge and cloud, scaling more effectively to large deployment scenarios.

Across these five subcategories, heuristic methods offer low overhead and interpretability, making them well-suited for resource-constrained edge deployments. However, individual heuristics typically target specific objectives (*e.g.*, latency, cost) and may require manual reconfiguration as workload patterns and network conditions change.

Learning-based. Placement and scheduling decisions in SEC must adapt to dynamic demand, volatile resources, and changing network conditions. RL is widely used because it handles such uncertainty while balancing multiple objectives such as latency, energy, and cost.

Federated RL approaches enable edge nodes to collaboratively learn placement policies. Vertical federated RL [165] uses proximal policy optimization (PPO)-based agents to score devices based on state and function characteristics, enabling QoS-aligned execution under high load while reducing violations and energy consumption. Hudson *et al.* [79] combine federated learning for request prediction with greedy placement and device-to-device collaboration, improving average QoS at the cost of relatively static resource assumptions.

Deep RL (DRL) methods directly optimize placement and scheduling decisions under rapidly changing conditions. Khansari and Sharifian [91] adopt actor-critic methods to determine function placement in distributed IoT environments, minimizing delay while balancing resource usage. ExpertDRL [212] applies deep deterministic policy gradient to jointly determine function placements and instance configurations across various hardware settings. Autonomic DRL placement with MAPE-K (Monitor – Analyze – Plan – Execute – Knowledge) control [211] employs priority queues and dynamic deployment decisions to reduce latency and cost under high request volumes. Li *et al.* [107] decompose the function placement problem into dispatching (where to send the function) and local scheduling (execution order) to meet SLOs under changing resource availability. Chen *et al.* [26] incorporate budget constraints into RL-based function placements.

Several DRL variants target specific placement requirements, including action-constrained DRL for delay and energy guarantees [213], Deep-Q methods for latency reduction [103], PPO-based systems for stability under dynamic load [83], multi-agent approaches for decentralized placement [106], and multi-objective RL for balancing completion time, energy, and cost [206].

Learning-based methods introduce adaptivity to placement and scheduling under dynamic and partially observable conditions, balancing multiple objectives without manually tuned rules. However, they typically require substantial training data, are prone to overfitting, and face scalability challenges as the number of services, nodes, and configuration spaces increases.

Optimization-based. Optimization-based approaches formulate placement and scheduling as constrained optimization problems, explicitly modeling resource constraints, latency targets, and locality constraints to obtain solutions with formal guarantees.

Integer programming-based methods (*e.g.*, Integer Linear Programming; ILP, Mixed Integer Program; MIP) provide exact solutions for placement problems. NEPTUNE [9] combines a multi-level control hierarchy with a MIP to place serverless functions close to users, minimizing network latency and resource consumption while ensuring service continuity. CANN [65] pairs ILP-based placement with an LSTM approximator trained on solver outputs, jointly optimizing bandwidth and computing resources while reducing convergence time by 95%.

Optimization formulations also make placement and scheduling decisions while balancing multiple objectives simultaneously. FOA [4] formulates placement as a multi-objective linear programming problem to optimize makespan, data locality, and cost across the edge-cloud continuum, using LP relaxation and rounding for near-optimal allocation. REPFS [21] places functions based on individual user sensitivity, using personality-driven QoS thresholds instead of

uniform latency targets. FaaSDeliver [210] determines function placement and resource allocation using Bayesian Optimization, reducing cost while meeting QoS across heterogeneous edge-cloud platforms.

Joint optimization approaches are also widely used to optimize placement along with related decisions, *e.g.*, caching and migration together, which are interdependent. Onco [76] jointly optimizes scheduling and container retention in vehicular networks, exploiting container image layer sharing to reduce costs and cold-start latency under high mobility. Huang *et al.* [77] use a predictive ILP-based framework with Receding Horizon Control to determine when and where to migrate functions, minimizing performance degradation as users move away from hosting nodes. DLDP [217] jointly optimizes scheduling and caching using dynamic programming, reducing loading latency by proactively pre-caching dependent libraries.

Beyond joint decisions, optimization approaches also adapt placement to diverse infrastructure configurations, including hybrid cloud-edge and multi-tier IoT deployments. Puliafito *et al.* [151] use a mixture of ILP to place functions and select an execution mode (stateful PaaS or stateless FaaS), minimizing state-access latency and cost. Pelle *et al.* [144] optimize placement by aligning decisions with real-time network telemetry, exploiting warm instances to enable seamless edge-to-cloud transitions. Apollo [177] optimizes placement for serverless compositions using a multi-objective evolutionary algorithm, enabling decentralized coordination to minimize latency and cost.

Batching optimization schedules multiple requests together, amortizing scheduling overhead and improving resource utilization. FDN [87] optimizes batch placement across Cloud, Edge, and HPC platforms using data-affinity-aware scheduling, reducing energy consumption by 17× while meeting SLOs. Tangram [145] adaptively sizes batches for serverless video analytics to maximize GPU utilization while meeting latency SLOs.

Optimization-based approaches offer principled mechanisms for handling constraints and can provide formal guarantees across diverse formulations, including ILP, multi-objective, and joint optimization techniques. However, they can be computationally intensive for large-scale deployments and depend on accurate system models, which may not fully capture runtime dynamics.

Online algorithms. In SEC, placement and scheduling decisions must be made as requests arrive without knowledge of future demand. Online algorithms address this by deciding when to retain or evict instances, balancing cold-start latency against memory consumption.

Retention of serverless instances directly affects scheduling by determining whether subsequent requests use warm instances or incur cold-start delays. Xiao *et al.* [200] propose online lazy caching to jointly optimize placement and scheduling for function chains, minimizing combined costs across heterogeneous edge nodes with a provable competitive ratio. S-Cache [28] uses LSTM-based prediction to dynamically prioritize instance retention, achieving 3× faster response times. Xia *et al.* [198] use Lyapunov optimization to dynamically switch between local and edge execution while adapting to user mobility.

Online approaches also jointly optimize placement, routing, and caching as requests arrive. FARM [97] formulates placement and scheduling as a constrained Markov decision process, balancing warm instance retention against memory constraints while meeting task deadlines. O-RDC [140] adaptively decides placement and container keep-alive duration using a ski-rental approach, balancing cold-start latency against resource costs under unpredictable workloads. Other approaches use greedy online algorithms to select among local, peer, and cloud execution paths [216], jointly optimize placement and data-flow routing with provable performance guarantees [174], or combine request distribution with container caching via Lyapunov optimization [27].

Moreover, online learning approaches address placement and scheduling under uncertain demand by learning from runtime observations rather than predefined models. Xu *et al.* [204] propose an age-aware online framework that jointly optimizes placement and scheduling to minimize Age of Information while ensuring resource efficiency, balancing data freshness against resource consumption in serverless edge clouds. Tütüncüoğlu *et al.* [188] optimize placement and scheduling under imperfect network information using online learning and dynamic game formulation.

Online algorithms make placement and scheduling decisions as requests arrive without prior knowledge of future demand. However, the lack of future visibility can result in suboptimal configurations compared to approaches with global knowledge (*e.g.*, offline methods).

Workflow-specific. Workflow-specific approaches schedule dependent functions in a DAG, considering execution order and inter-stage communication to minimize end-to-end latency.

Workflow modeling approaches determine scheduling decisions by defining how dependencies and contention are handled. BeeFlow [120] optimizes placement and scheduling for behavior-tree workflows using contention-aware partitioning, identifying non-concurrent tasks and distributing them across nodes to achieve 3.2× speedup. Storm-RTS [136] places serverless functions for stream processing across cloud-edge sites based on declarative policies and schedules, invoking them at guaranteed rates, maintaining stable throughput despite edge heterogeneity.

Dynamic optimization methods address the joint placement and scheduling of DAG tasks (*i.e.*, serverless function chains) to achieve diverse system objectives, including deadline satisfaction, completion time, energy efficiency, and cost. For example, Xie *et al.* [202] employ D3QN to optimize IIoT DAG workflows, adaptively handling dynamic conditions and inter-function dependencies. DPE [43] jointly optimizes placement and scheduling for a stream of serverless functions by splitting data across heterogeneous execution paths, achieving up to a 40% reduction in makespan compared to traditional heuristics. Similarly, Sdser [116] leverages Bayesian prediction to proactively adjust placement and reschedule tasks at runtime, maximizing container reuse and reducing overhead by 27% for dynamic DAG workflows.

Workflow-specific methods effectively exploit task dependencies to efficiently place and schedule serverless functions. However, coordination complexity increases as the DAG/serverless functions grow, and most approaches assume static DAG structures that may not reflect runtime variability.

3.3 Dynamic Function Offloading

Dynamic offloading decides at runtime whether to execute functions locally at the edge or offload them to peer edge nodes or cloud, adapting to changing workload and network conditions. These decisions can be made by centralized orchestrators or distributed across nodes, each with trade-offs between coordination complexity and scalability.

3.3.1 Current approaches. We organize existing approaches into three categories: (i) decentralized offloading, which distributes decision-making across nodes to avoid centralized bottlenecks; (ii) edge-cloud coordination, which synchronizes offloading decisions across tiers; and (iii) learning-based offloading, which adapts decisions under dynamic and uncertain conditions. Table 5 compares the strengths and limitations of each category.

Decentralized offloading. Decentralized offloading distributes decision-making across nodes to avoid centralized bottlenecks and single points of failure. Serverledge [122] organizes edge and cloud nodes into zones and regions, enabling horizontal offloading to peer edge nodes or vertical offloading to higher tiers based on load. Cicconetti *et al.* [35, 40] propose decentralized dispatching architectures where each dispatcher forwards function requests to edge hosts based on network and computational latency using only local information, achieving stable response times in high-mobility scenarios. Escobar *et al.* [46] replace centralized brokers with a peer-to-peer dataflow architecture using

Table 5. Comparison of dynamic function offloading approaches.

Sub-Category	Strengths	Limitations	Surveyed Work
Decentralized offloading	Improves scalability and fault tolerance by eliminating central coordination points	May lead to suboptimal decisions when nodes lack visibility into system-wide state	[35, 40, 122] [46]
Edge-cloud coordination	Adapts offloading decisions based on response times, data locations, and workload characteristics	Effectiveness depends on accurate runtime monitoring and low cross-tier communication latency	[128, 161, 176] [32, 96, 196] [117, 143, 146]
Learning-based offloading	Adapts to dynamic conditions without explicit models	Learned policies may not transfer effectively across diverse SEC env.	[183, 192, 198] [202]

the Zenoh protocol, enabling lightweight nanoservices to execute across the cloud-to-edge continuum with significantly reduced round-trip latency.

Decentralized offloading approaches can improve scalability and fault tolerance by eliminating central coordination points. However, they face challenges in maintaining global consistency and may lead to suboptimal decisions when nodes lack visibility into system-wide state.

Edge-cloud coordination. Edge-cloud coordination synchronizes offloading decisions across tiers to balance latency, resource utilization, and data locality. Knative Edge [176] enables location-agnostic function placement by mirroring service definitions across the edge-cloud continuum and adaptively offloading requests based on real-time response-time ratios. Similarly, Risco *et al.* [161] employ a dual rescheduling strategy that proactively relocates functions when local resources are scarce and reactively offloads to the cloud when edge queuing exceeds a threshold. Context-aware approaches further consider inter-function dependencies. For example, Marchese and Tomarchio [128] jointly optimize placement and scheduling based on infrastructure constraints and communication patterns, while Ko *et al.* [96] partition DNN models across edge nodes based on network conditions and resource availability.

Beyond inter-function dependencies, data location also affects offloading decisions. Data-aware offloading minimizes data transfer by executing computation close to data sources. When data locations change dynamically, Fog Function [32] migrates functions between edge and cloud to follow data availability, and CSPOT [196] provides a portable runtime that flexibly moves computation to data or data to computation across the continuum. Application-specific platforms tailor offloading to workload characteristics. For example, Serverless CEI [117] distributes ML workloads hierarchically with opportunistic scheduling that balances accuracy against latency, HiveMind [143] offloads resource-intensive UAV swarm computations to the cloud while processing time-critical tasks locally, and Kappa [146] dynamically maps actors across edge and cloud nodes based on hardware capabilities.

These approaches adapt offloading decisions based on response times, data locations, and workload characteristics, but their effectiveness depends on accurate runtime monitoring and low cross-tier communication latency.

Learning-based offloading. Learning-based offloading adapts decisions under dynamic and uncertain conditions using prediction models, optimization theory, and reinforcement learning. LaSS [192] predicts response times using a processor-sharing model and schedules functions based on SLO priorities across the edge-cloud continuum, ensuring latency-critical tasks meet deadlines under cold-starts and contention. Xia *et al.* [198] combine Lyapunov optimization with game-theoretic modeling to enable adaptive offloading decisions in MEC-enabled ultra-dense networks without requiring global system state. Tang *et al.* [183] formulate task scheduling as a partially observable stochastic game and use multi-agent deep reinforcement learning to enable decentralized scheduling based on local observations, balancing

delay, energy, and queue stability. For workflow-based applications, Xie *et al.* [202] model IIoT workflows as DAGs and use deep reinforcement learning to decide whether to execute functions locally or offload to the cloud based on real-time channel conditions, jointly optimizing execution time, energy, and cost.

Learning-based offloading adapts to dynamic conditions but learned policies may not transfer effectively across diverse edge environments.

3.4 Multi-Tenancy and Isolation

Multi-tenancy at the edge differs from cloud multi-tenancy because resources are constrained and deployments typically span only a single node or a small cluster. In edge settings, multiple tenants (users, services, or function groups) share the same hardware, making software-level isolation essential for security, performance predictability, and lifecycle management. While cloud serverless platforms commonly use microVMs (*e.g.*, Firecracker [2], gVisor [209]), SEC systems on edge devices typically rely on containers, with emerging alternatives (*e.g.*, unikernels [132, 134] and WebAssembly [53, 139]) aiming to reduce overhead and improve startup latency.

3.4.1 Current Approaches. Table 6 compares and summarizes their unique characteristics.

Table 6. Comparison of Isolation Mechanisms and Workflows for SEC

	Containers	**Unikernels**	**WebAssembly**
Examples	Docker, containerd	Nanos, OSv	Wasmtime, WasmEdge
Isolation	OS-level (namespace, cgroups); shared kernel	Single-address-space; minimal OS	Sandbox runtime; memory isolation
Startup time	100 – 500 ms	10 – 50 ms	1 – 10 ms
Resource footprint	Low (shared kernel)	Very low (minimal image)	Very low (compact modules)
Maturity	Widely adopted	Less mature; limited tooling	Emerging, ecosystem developing

Containers. Containers are the dominant isolation mechanism for SEC and serverless platforms in general. When a serverless function is triggered by a user request, the platform pulls a container image locally or from an image registry to spin up an on-demand container environment that shares the host OS's kernel. After function execution completes, container instances remain idle (warm) waiting for subsequent requests; if no request arrives within a certain period of time (container timeout), the container is terminated (freeze/thaw cycle). The lightweight nature of containers, combined with wide compatibility and support for high concurrency, has made them the default choice in most major open-source frameworks, including OpenFaaS[3], OpenWhisk[4], and Knative[5]. Container isolation using namespaces and cgroups can limit cross-function memory/file access [193], but per-request container creation incurs 100ms – 500ms of startup latency, which can negatively impact the performance of latency-sensitive workloads. Therefore, SEC systems rely on container reuse strategies, *e.g.*, warm pools [67], long-lived containers [148], to reduce overhead. Despite their maturity, containers share the host kernel, which limits isolation strength.

[3]https://www.openfaas.com/
[4]https://openwhisk.apache.org/
[5]https://knative.dev/

Unikernels. Unikernels build minimal single-address-space virtual machine images tailored to a specific application. When a serverless function is triggered, the serverless platform loads the corresponding unikernel image and boots it on a hypervisor for execution. The image bundles application code with a minimal OS (library OS) as a single binary, containing only the necessary services required by the workload. This lightweight image runs directly on a hypervisor (*e.g.*, Xen), providing strong isolation through hardware-level virtualization [132]. Moebius *et al.* [134] compare unikernels (*e.g.*, Nanos, OSv) against containers and microVMs, finding that unikernels achieve low memory footprint and fast cold-start performance (typically 10ms – 50ms), making them attractive for latency-sensitive SEC scenarios. However, the ecosystem remains less mature than containers, with challenges in Linux compatibility, tooling, and developer experience [134].

WebAssembly. WebAssembly (Wasm) is a lightweight, sandboxed environment that is growing as a promising technology for serverless workloads. Upon function invocation, the serverless platform's runtime loads the `.wasm` module and instantly creates an isolated sandbox environment. The sandbox enforces memory boundaries at runtime, providing isolation without relying on kernel mechanisms. The WebAssembly System Interface (WASI) enables portable interaction with operating system resources without compromising security. Wasm runtimes offer significant cold-start improvements over containers, with studies reporting reductions of up to 99.5% on resource-constrained devices [53] and module instantiation times in the microsecond range [54]. The compact size of Wasm modules allows them to be preemptively distributed to edge workers, reducing cold-start latency compared to pulling large container images on demand [139]. These properties enable high-density, multi-tenant computation with lower latency than container-based runtimes in edge deployments [54]. However, Wasm currently has limited language support and restricted system call capabilities. Consequently, Wasm runtimes are particularly well-suited for low-end edge hosts such as Raspberry Pis, where they significantly outperform container-based approaches [53]. For more capable edge nodes, hybrid approaches that use Wasm as the initial responder to mitigate cold-start latency while lazy-loading containers for compute-intensive tasks can reduce tail latency and improve memory utilization [71].

Isolation techniques in SEC – *container*, *unikernel*, and *Wasm* – are fundamentally a three-way tradeoff among security strength, startup latency, and steady-state footprint. No single approach dominates, as the choice depends on security requirements, performance constraints, and ecosystem maturity. Systematic frameworks for selecting and combining isolation approaches based on workload characteristics remain an open challenge.

3.5 Energy-Aware Resource Management

Serverless edge nodes and devices often operate under strict power constraints. They often run on batteries or limited power supplies, or they can harvest energy from renewable sources (*e.g.*, solar, wind), introducing power supply uncertainty and storage limitations. A growing body of work addresses these constraints by incorporating energy into resource management decisions.

3.5.1 Current approaches. Energy-aware scheduling optimizes function placement and execution under power constraints, addressing both fixed power budgets and the additional uncertainty of variable renewable supply.

3GC [14] applies DVFS (Dynamic Voltage and Frequency Scaling) to adapt CPU frequency during function execution on edge nodes, minimizing energy and cost while meeting latency SLOs. COME [1] optimizes energy through a two-timescale approach, using threshold-based container provisioning to reduce cold-starts while dynamically scaling resources based on load. Patel and Townend [142] propose a decentralized stable matching-based scheduling framework that prioritizes placing serverless functions on nodes with higher renewable energy availability across the edge-cloud

continuum. faasHouse [5] dynamically matches workload to energy availability across battery-powered and renewable-energy edge nodes, migrating functions to prevent failures on energy-depleted nodes while utilizing surplus energy on others. Similarly, LEASE [190] reduces energy consumption by offloading functions from high-load nodes to low-load nodes, calculating net energy savings to select the most energy-efficient placement while meeting service deadlines, and Aslanpour *et al.* [7] monitor battery state of charge on renewable-powered edge nodes and proactively migrate functions from energy-depleting nodes before they fail, ensuring service continuity and improving cluster availability. Beyond placement, Calavaro *et al.* [20] extend the Serverledge platform [163] with approximate computing, using hardware-level energy monitoring (Intel RAPL) and a dual-threshold policy to switch to lighter function variants at moderate battery levels and offload tasks to neighboring edge nodes or the cloud when energy becomes critical. Vahabi *et al.* [189] predict workload patterns to consolidate function executions on minimal edge nodes and proactively power down idle workers, achieving 12% – 45% energy savings over baseline heuristics. For energy-harvesting environments, Li *et al.* [112] use pricing mechanisms to shift function execution toward periods of high renewable availability, reducing grid dependence.

Existing energy-aware approaches demonstrate that incorporating energy state into placement and execution decisions can materially reduce consumption and improve availability. However, most studies optimize energy in isolation from other objectives such as cold-start mitigation and latency, and few have been validated on real battery-powered or renewable-energy edge deployments.

3.6 State Management

The stateless FaaS execution model requires functions to store persistent state in remote storage. This stateless nature may introduce significant latency overhead in distributed edge environments where every state access incurs network round trips. Below, we survey how SEC research addresses this challenge through state locality and placement strategies.

3.6.1 Current approaches. Cloud serverless research [86, 179] has established common approaches for stateful functions, *e.g.*, state co-location, runtime-level sharing, durable coordination, but these designs assume resource-abundant environments with fast, reliable networks. SEC platforms must adapt these ideas under limited bandwidth, heterogeneous hardware, and intermittent connectivity. Indeed, experimental evaluation confirms that deploying stateful serverless frameworks designed for cloud environments directly at the edge leads to significant performance degradation [74].

Cicconetti *et al.* [37] propose and compare three state management models for edge function chains: `PureFaaS`, which relies on remote storage; `StateProp`, which propagates the entire application state through function arguments; and `StateLocal`, which keeps state local to edge workers and passes only references between invocations. Simulation results show that the StateLocal approach significantly reduces end-to-end latency and network traffic. A follow-up study [38] validates these findings on a decentralized prototype (*ServerlessOnEdge*) and introduces virtual-edge mechanisms to maintain causal consistency in DAG workflows, confirming that anchoring state locally is essential when embedding full state data into function invocations becomes prohibitive at scale. The same group further generalizes the design space into four architectural models, *i.e.*, `External`, `In-edge`, `In-function`, and `In-client`. The authors identify the `In-client` model, where clients pass state directly through function arguments and receive updated state in return values, as an effective approach for SEC that achieves low latency without requiring serverless container migration [39]. Wen *et al.* [195] introduce LoLa, a framework that adaptively co-locates state with functions on edge nodes and dynamically migrates state based on scheduling decisions and network conditions. Similarly, Capybara [30] provides a distributed object store for edge serverless that uses DHT-based P2P architecture and programmable handlers to co-locate function data with invocations while supporting locality-aware replication. At a coarser granularity, Puliafito

Table 7. Strengths and limitations of evaluation tools for SEC resource management.

Category	Sub-Category	Strengths	Limitations	Tools
Benchmark	Function-level	Direct measurement of SEC metrics (cold-start, autoscaling, throughput) on real edge HW	Measures isolated functions; may not reflect end-to-end app or workflow perf.	EdgeFaaSBench [159], Kjorveziroski *et al.* [94], FunLess [139]
	Application & Workflow	Evaluate representative application scenarios. *e.g.*, DL-inference/multi-step pipelines	Workload-specific; findings may not generalize across domains	ESBench [29], OSCAR-P [55], EdgeBench [205]
	Cross-deployment	Compare performance across edge-cloud tiers; reveal deployment-specific trade-offs	Sensitive to deployment-specific configurations; hard to generalize	Carpio *et al.* [22], FDNInspector [87], ComFaaS Dist. [105], Raith *et al.* [158], Umbilical Choir [126]
Simulation/ Emulation	–	Low-cost, large-scale policy exploration under controlled conditions	Accuracy depends on model fidelity; may not capture real hardware and net. behavior	FogFaaS [59], `faas-sim` [157], SimuScale [156]
Testbed	–	Realistic experimentation on real heterogeneous hardware and network conditions	Scarce; limited coverage of device, accelerator, and network configurations	CHI@Edge [90], BenchFaaS [23]

et al. [151] model the trade-off between local state persistence at the edge and remote stateless invocations to the cloud, providing a cost optimization framework that decides when to keep stateful containers at the edge versus dispatching calls to cloud storage.

Existing work demonstrates that SEC state management is converging on locality-aware designs that keep state close to compute through local storage, adaptive co-location, and intelligent edge-cloud allocation. However, most approaches have been validated only in controlled settings, and few address realistic edge dynamics (*e.g.*, node mobility, intermittent connectivity). Bridging state locality with consistency guarantees under these conditions remains open (see §5.2).

3.7 Benchmark, Evaluation Tools and Testbeds

Evaluating resource management in SEC poses unique evaluation challenges, as realistic assessment must capture cold- and warm-starts, autoscaling dynamics, multi-tenancy interference, heterogeneous hardware (*e.g.*, CPU, GPU, accelerators), and non-ideal network conditions (*e.g.*, bandwidth limits, intermittent connectivity). Since these factors are increasingly difficult to reproduce at the edge than in resource-abundant cloud environments, the research community relies on three complementary evaluation tools: benchmark suites, simulation/emulation frameworks, and testbeds.

3.7.1 Current approaches. These three tools are complementary. Benchmark suites offer standardized measurement tools and serverless workloads on real edge, simulators enable large-scale evaluation of resource management policies under controlled conditions and testbeds provide real-world heterogeneous edge-to-cloud infrastructure for experimentation. Table 7 compares the strengths and limitations of these three classes.

Benchmark suites. Benchmarking serverless platforms at the edge spans a wide range of evaluation targets, from platform-level metrics to application-specific workloads and cross-tier deployment comparisons. We group existing efforts into three categories: (i) function-level benchmarks that measure platform metrics across heterogeneous devices,

(ii) application and workflow benchmarks that target specific workload types (*e.g.*, DNN inference, DAG pipelines), and (iii) cross-deployment benchmarks that compare performance across edge-cloud tiers.

(i) *Function-level benchmarks* measure platform metrics, *e.g.*, cold-start latency, autoscaling behavior, throughput, and resource utilization, across heterogeneous edge devices. EdgeFaaSBench [159] is an open-source benchmark suite built on OpenFaaS that evaluates serverless performance on heterogeneous edge devices (*e.g.*, Raspberry Pi, Jetson Nano), measuring cold- and warm-starts, concurrent execution, and resource utilization across CPU and GPU workloads. Kjorveziroski *et al.* [94] adapt the cloud-oriented FunctionBench suite [92] to evaluate edge-optimized Kubernetes distributions (K8s, K3s, and MicroK8s) on bare-metal servers as edge nodes. FunLess [139] evaluates a WebAssembly-based serverless platform against container-based alternatives such as OpenFaaS and Knative on resource-constrained devices, demonstrating that WASM can extend serverless capabilities to extreme edge environments where standard Kubernetes orchestration is infeasible.

(ii) *Application and workflow benchmarks* evaluate SEC performance under representative SEC application scenarios rather than isolated function invocations. ESBench [29] benchmarks deep learning inference on mobile-edge networks using OpenFaaS, measuring how resource contention, model complexity, and network conditions (3G/4G/5G/Wi-Fi) affect inference latency. For workflow-oriented evaluation, OSCAR-P [55] automates performance profiling of SEC workflows across heterogeneous edge-cloud configurations, benchmarking multi-component applications such as mask detection on hardware ranging from edge device clusters to cloud VMs. EdgeBench [205] evaluates representative serverless workflows (*e.g.*, video analytics, IoT pipelines) across edge-cloud tiers, enabling bottleneck identification at both function and workflow levels.

(iii) *Cross-deployment benchmarks* evaluate performance across deployment tiers. Carpio *et al.* [22] evaluate serverless functions across an edge-cloud continuum spanning VMs, Raspberry Pis, and cloud datacenters, finding that proximity benefits lightweight functions but the lack of network-aware scheduling leads to unpredictable performance. FDNInspector [87] is an open-source tool that benchmarks individual serverless functions across heterogeneous platforms including edge devices, cloud VMs, and HPC clusters, collecting tiered metrics such as response times and resource utilization to inform SLO-compliant scheduling. ComFaaS Distributed [105] benchmarks parallel FaaS execution using MPI across edge and cloud nodes at varying geographic distances, showing that edge latency advantages grow with distance but parallelization overhead can outweigh gains for smaller tasks. Raith *et al.* [158] provide an end-to-end benchmarking framework based on Galileo for evaluating cluster management techniques across the edge-cloud continuum, supporting distributed experiments on heterogeneous CPU architectures with automated orchestration and resource monitoring. Beyond static benchmarking, Umbilical Choir [126] is an open-source, geo-aware live-testing framework that automates complex release strategies like A/B testing and canary rollouts for serverless applications across the edge-cloud continuum.

These benchmarks span evaluation targets from function-level metrics to cross-tier deployment comparisons, however, no single study covers the full range, and the lack of standardized metrics and configurations hinders reproducible comparison.

Simulation/Emulation Tools. Simulation and emulation tools provide a low-cost alternative to physical deployments, enabling large-scale evaluation of resource management policies under controlled conditions. FogFaaS [59] extends the iFogSim2 simulator [123] with dedicated modules for cold-start modeling, autoscaling, scheduling, and cost analysis under fog and edge scenarios. `faas-sim` [157] is an open-source, trace-driven discrete-event simulation framework for serverless edge platforms, treating scaling, placement, and routing as first-class entities using real-world profiling data.

SimuScale [156] builds on faas-sim to enable co-simulation-driven autoscaling optimization, using live system state to run what-if scenarios and reduce SLO violations in serverless edge deployments. However, the accuracy of these tools depends on model fidelity, and most existing simulators may not capture real hardware and network behavior.

Testbeds. Testbeds provide experiment infrastructure with real heterogeneous hardware and network conditions. CHI@Edge [90] extends the Chameleon Cloud platform to support edge-to-cloud experiments with flexible topologies and user-contributed devices, providing a general-purpose testbed that SEC researchers can leverage for heterogeneous deployment studies. BenchFaaS [23] is an open-source network testbed built on K3s and OpenFaaS that automates serverless benchmarking across heterogeneous hardware (AMD64, ARM64) under emulated network conditions such as WAN delay and packet loss, providing both function-level and workflow-level metrics. However, SEC testbeds remain scarce, and existing offerings may cover only a narrow range of the device, accelerator, and network configurations found in real deployments.

4 Platform Support for SEC Resource Management

SEC research typically extends open-source platforms (OpenFaaS, OpenWhisk, Knative) or commercial offerings (AWS Greengrass[6], Azure IoT Edge[7]).

OpenFaaS. OpenFaaS is an open-source serverless framework that supports multiple orchestrators, including Kubernetes, Docker Swarm, and `faasd`[8] (a lightweight containerd-based standalone option). The latter two offer simpler orchestration with lower resource overhead, but are best suited for single-node or small-cluster deployments due to limited scalability and auto-scaling capabilities. SEC research leveraging OpenFaaS spans autoscaling, energy efficiency, and resource placement. Representative work includes RL-driven autoscaling configuration [18], cost-benefit-based replica adjustment under bursty demand (KneeScale) [110], DVFS-integrated energy-aware allocation (3GC) [14], and multi-resource placement with GPU support (NEPTUNE) [9]. OpenFaaS has also been deployed on renewable-powered Raspberry Pi clusters to study QoS under battery constraints [7] and to characterize cold- and warm-start behaviors under low-resource conditions [17].

Apache OpenWhisk. Apache OpenWhisk offers both a full deployment and a Lean variant. The Lean variant significantly reduces memory footprint, making OpenWhisk feasible on resource-constrained edge devices. SEC research using OpenWhisk focuses on decentralized execution and latency-sensitive workloads, such as decentralized in-network task dispatch for short- and long-term fairness [36], reward-driven scheduling with real-time queue monitoring (PROBA) [141], peer-to-peer discovery on volunteer nodes (FaaS@Edge) [62], and queueing-theoretic latency prediction with separated control and data paths (LaSS) [192]. OpenWhisk has also been extended with fog-layer invokers and edge-local object stores (Minio) to cache data and reduce transfer overhead during function initialization [10].

Knative. Knative provides serverless abstractions on Kubernetes, including request-driven autoscaling and event routing. SEC research extends Knative's autoscaling, scheduling, and routing mechanisms for edge-cloud environments, including queue-metric-based autoscaling and load balancing (Mu) [133], shared memory for DNN inference [93], network-aware scheduling for distributed deployments [128], edge-cloud offloading based on response times [176], and eBPF-based lightweight proxies for IoT traffic [193].

[6] https://aws.amazon.com/greengrass/
[7] https://azure.microsoft.com/en-us/products/iot-edge
[8] https://github.com/openfaas/faasd

Globus Compute. Globus Compute [13], incorporating funcX [114], provides federated function execution primarily targeting scientific and HPC workloads. Users register functions with a central cloud service and specify which endpoint should execute them. Endpoints can be deployed on HPC clusters, edge devices, or other sites.

Commerical platforms. AWS Greengrass and Azure IoT Edge follow a hybrid model where functions execute locally on edge devices while cloud services handle orchestration, analytics, and storage. However, these platforms are not open-source, limiting their use as research platforms. Separately, CDN-based services, *e.g.*, Lambda@Edge and Cloudflare Workers, run on provider-managed infrastructure with abundant resources and always-on connectivity, making them network edge platforms rather than SEC solutions.

Despite the availability of these platforms, several fundamental challenges in SEC resource management remain unresolved, as discussed in §5.

5 Open Challenges and Future Research Directions

This section identifies open problems and future research directions that emerge from the surveyed SEC literature. We focus on four cross-cutting challenges that are rapidly becoming central to SEC but remain insufficiently supported by today's platforms. These include (i) emerging AI and large language model (LLM) (§5.1), (ii) stateful operations under edge constraints (§5.2), (iii) carbon-aware resource management (§5.3), and (iv) reproducible evaluation standards (§5.4). We then summarize additional challenges including cold-starts, image distribution, capacity-bound scaling, long-running functions, and performance variability (§5.5).

5.1 LLM-Scale Inference at the Edge

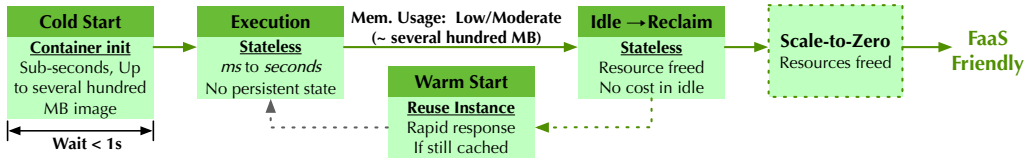
SEC workloads are increasingly dominated by AI inference, including DNN pipelines and LLM serving. These workloads are memory-intensive, state-heavy (*e.g.*, KV caches), multi-stage, and exhibit bursty demand. Even in resource-rich cloud environments, memory management, scheduling, and state migration are first-order performance bottlenecks [101, 115, 181]. The standard FaaS model, which is short-lived, stateless, scale-to-zero, *may be ill-suited* for such workloads, and edge-specific constraints (*e.g.*, limited resources, heterogeneous accelerators) further complicate the problem, preventing existing cloud solutions from directly translating to SEC.

Figure 5 contrasts these two lifecycle patterns. A typical serverless function completes in sub-seconds and releases resources immediately, making scale-to-zero straightforward. LLM inference, however, requires loading multi-GB model weights and maintaining a stateful KV-cache throughout token generation. This creates a scale-to-zero dilemma where keeping the model resident occupies scarce edge memory, while reclaiming it incurs prohibitive cold-start latency on the next request.

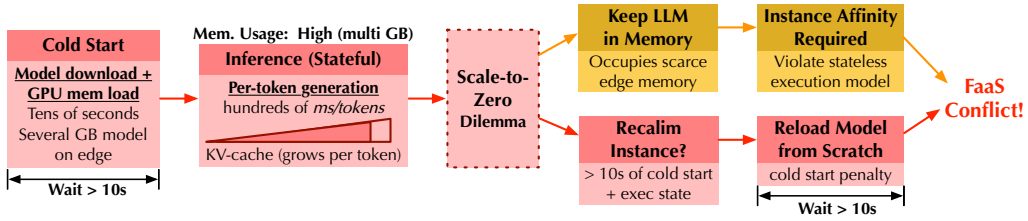
Open problems and future directions. LLM inference in SEC faces four key challenges, each suggesting distinct research directions.

(1) *Cold-start and model distribution.* Model binaries and checkpoints can be multi-GB, and bursty demand requires rapidly bringing up additional replicas. Edge networks amplify distribution costs, motivating decentralized and locality-aware model distribution. Edge-optimized small language models offer a complementary path by reducing model size and memory footprint [119].

(2) *Memory-intensive execution and KV-cache management.* LLM inference benefits from persistent caches and memory reuse, but the SEC scale-to-zero model works against this. KV caches behave like high-churn state tightly coupled to scheduling and routing decisions [181]. Model- and accelerator-aware orchestration should



(a) Standard serverless function lifecycle at the edge.



(b) LLM inference lifecycle at the edge.

Fig. 5. Comparison of a typical serverless function lifecycle (a) versus LLM inference lifecycle (b) at the edge. *(a) Standard serverless functions are lightweight, stateless, and incur sub-second cold-start overhead, aligning well with the FaaS model. (b) LLM inference requires loading multi-GB models and maintaining stateful KV-caches, creating a scale-to-zero dilemma that conflicts with the FaaS model.*

become a first-class SEC capability, with schedulers modeling GPU/TPU/NPU availability and multi-tier memory [52, 101].

(3) *Edge-cloud split inference.* Partitioning models or requests between edge and cloud can trade latency, bandwidth, privacy, and cost, but requires online control under variable network conditions [208]. Split inference must account for cross-node communication costs and burstiness [85].

(4) *Multi-step pipelines and workflow orchestration.* Many LLM applications resemble workflow execution rather than independent invocations. Dependency-aware placement, pipeline-aware prewarming, and DAG-aware autoscaling can align multi-step pipelines with edge locality and capacity constraints [52, 115]. Hybrid serverless-container deployments, *e.g.*, stateful containers for model/KV-cache residency, serverless functions for event-driven glue logic, can offer a pragmatic model for such workloads.

5.2 Stateful Operations in SEC

As SEC applications evolve from isolated event handlers to pipelines and interactive services, application state management becomes a core requirement. This includes user/session state, caching metadata, coordination state, and model/KV state. While §3.6 surveys current approaches to state locality and placement, several fundamental challenges remain open.

Open problems and future directions. Stateful SEC faces three key challenges, each suggesting distinct research directions.

(1) *State storage location.* Current work has established that keeping state local to edge workers reduces latency (§3.6), but practical SEC deployment raises unresolved trade-offs. For example, colocating functions with their dominant state can reduce cross-node traffic, but *may* lead to load imbalance and complicate scheduling decisions. How to balance locality benefits against resource constraints and scheduling flexibility remains underexplored.

(2) *State synchronization.* Edge nodes may disconnect, move, or fail, and the state must remain usable under these conditions. Designing synchronization mechanisms that tolerate intermittent connectivity and network partitions remains an open challenge.
(3) *Recovery after failure.* When an edge node fails, recovery must be fast and minimize state transfers that could delay function restarts or cause network congestion.

5.3 Carbon-Aware Resource Management

Reducing energy consumption does not necessarily reduce carbon emissions. The same energy use can have very different CO_2 impact depending on the grid's carbon intensity. SEC complicates this because edge deployments span multiple locations with different carbon intensities, and placement decisions affect both latency and emissions [178, 197]. Yet unlike cloud providers that increasingly adopt carbon-aware workload shifting [154], SEC platforms lack carbon visibility at fine spatial granularity and have no mechanisms to act on carbon signals. Most SEC resource management work focuses on latency and device energy, rarely optimizing directly for carbon footprint.

Open problems and future directions. Carbon-aware SEC faces three key challenges, each suggesting distinct research directions.

(1) *Carbon-aware scheduling and workload shifting.* Carbon intensity varies across regions and time, and renewable-powered edge nodes add uncertainty in energy supply. Shifting workloads to lower-carbon locations or times can reduce emissions while respecting latency SLOs [178, 197], but integrating carbon signals with energy availability into SEC schedulers remains an open challenge [180].
(2) *Carbon-aware metrics.* SEC lacks standardized metrics, *e.g.*, CO_2 per invocation or carbon-normalized SLO compliance, making it difficult to compare systems. Energy-efficiency and carbon-awareness can diverge [218], motivating new sustainability metrics that capture both dimensions.

5.4 Benchmarking and Reproducibility

The evaluation methodologies surveyed in §3.7 reveal persistent gaps in coverage and standardization. No benchmark suite has become a community standard, testbeds are limited, and public traces from real SEC deployments are lacking. Moreover, workload assumptions are often unrealistic, with many studies using synthetic arrivals and single-function microbenchmarks rather than chained workflows or AI inference pipelines [22, 156, 159]. Evaluation metrics also vary widely, limiting comparability across studies.

Open problems and future directions. Reproducible SEC evaluation faces three key challenges, each suggesting distinct research directions.

(1) *Lack of public SEC traces.* Unlike cloud serverless, where public traces are available [78, 80, 130, 173], SEC lacks comparable real-world traces. Collecting and publishing such traces would enable trace-driven simulation for reproducible evaluation [157].
(2) *Limited benchmark coverage.* Existing SEC benchmarks rarely cover multi-function workflows, accelerator-heavy inference, or carbon-aware metrics. A community-standard SEC benchmark suite should integrate function-level and workflow benchmarks, controlled network and power impairment, and standardized metrics for cold-start, SLO compliance, and carbon accounting.
(3) *Standardized evaluation methodology.* Comparing policies across different devices requires normalized metrics and controlled testbed conditions. Consolidating existing testbeds and publishing evaluation checklists (workload, metrics, hardware, configuration) should become standard practice in SEC research [23, 90].

5.5 Additional Challenges and Open Questions

Beyond the challenges above, several open questions remain in SEC systems research.

Cold-start and image distribution at the edge. Snapshotting and restore techniques (*e.g.*, initialization-less booting, checkpoint/restore) can reduce cold-starts [44, 104], but edge deployments add constraints on storage and bandwidth. Distribution of container images for function instances is slower and less reliable at the edge, and decentralized or topology-aware distribution with improved layer reuse can reduce redundant transfers [50].

Capacity-bound autoscaling and long-running functions. SEC is bounded by physical capacity, requiring capacity-aware autoscaling that integrates admission control and offloading to prevent SLO violations under bursty arrivals. Additionally, many emerging workloads (*e.g.*, enAI pipelines) exceed typical serverless timeouts, motivating hybrid models that combine serverless triggers with long-lived services [179].

Runtime/performance characterization of SEC in real-world deployments. Edge device performance varies with power availability, temperature, hardware heterogeneity, and co-located workload interference, yet systematic measurement of these effects on real edge deployments remains limited.

6 Conclusion

The transition from cloud serverless to SEC fundamentally changes the assumptions underlying resource management. This paper surveyed how the SEC community is addressing these challenges through a unified taxonomy covering autoscaling, placement, scheduling, offloading, isolation, energy-aware management, state management, and evaluation methodologies. We identified three recurring themes. First, effective SEC control is inherently multi-objective: optimizing latency alone can increase energy or cost, and aggressive caching can degrade fairness under scarcity. Second, heterogeneity-awareness is essential, as platforms must explicitly model diverse hardware capabilities, including specialized accelerators, rather than treating nodes as generic compute units. Third, most existing approaches optimize individual resource management dimensions in isolation, yet scaling, placement, isolation, and energy decisions are interdependent. Progress toward holistic, cross-dimension control remains limited.

Despite this progress, significant gaps remain. Evaluations are fragmented by inconsistent metrics and limited public artifacts, stateful execution and accelerator-aware orchestration are not yet first-class concerns in most SEC stacks, and emerging workloads such as LLM inference push beyond the assumptions of current platforms. Closing these gaps will require integrated designs that jointly consider platform architecture, runtime control, and workload characteristics.

References

[1] Madhura Adeppady, Alberto Conte, Paolo Giaccone, Holger Karl, and Carla-Fabiana Chiasserini. 2025. Dynamic Management of Constrained Computing Resources for Serverless Services. *IEEE Transactions on Network and Service Management* 22, 2 (2025).

[2] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. 2020. Firecracker: Lightweight Virtualization for Serverless Applications. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

[3] Deafallah Alsadie. 2024. A Comprehensive Review of AI Techniques for Resource Management in Fog Computing: Trends, Challenges, and Future Directions. *IEEE Access* 12 (2024), 118007–118059.

[4] Luc Angelelli, Anderson Andrei Da Silva, Yiannis Georgiou, Michael Mercier, Grégory Mounié, and Denis Trystram. 2023. Towards a Multi-objective Scheduling Policy for Serverless-based Edge-Cloud Continuum. In *IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*.

[5] Mohammad Sadegh Aslanpour, Adel Nadjaran Toosi, Muhammad Aamir Cheema, and Mohan Baruwal Chhetri. 2024. Faashouse: Sustainable Serverless Edge Computing Through Energy-Aware Resource Scheduling. *IEEE Transactions on Services Computing* 17 (2024).

[6] Mohammad Sadegh Aslanpour, Adel Nadjaran Toosi, Muhammad Aamir Cheema, Mohan Baruwal Chhetri, and Mohsen Amini Salehi. 2024. Load balancing for heterogeneous serverless edge computing: A performance-driven and empirical approach. *Future Gener. Comput. Syst.* (2024).

[7] Mohammad Sadegh Aslanpour, Adel Nadjaran Toosi, Muhammad Aamir Cheema, and Raj Gaire. 2022. Energy-Aware Resource Scheduling for Serverless Edge Computing. In *IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*.

[8] Mohammad Sadegh Aslanpour, Adel Nadjaran Toosi, Claudio Cicconetti, Bahman Javadi, Peter Sbarski, Davide Taibi, Marcos Dias de Assunção, Sukhpal Singh Gill, Raj Gaire, and Schahram Dustdar. 2021. Serverless Edge Computing: Vision and Challenges. In *Australasian Computer Science Week Multiconference (ACSW)*.

[9] Luciano Baresi, Davide Yi Xian Hu, Giovanni Quattrocchi, and Luca Terracciano. 2024. *NEPTUNE*: A Comprehensive Framework for Managing Serverless Functions at the Edge. *ACM Trans. on Autonomous and Adaptive Sys.* 19 (2024).

[10] Luciano Baresi and Danilo Filgueira Mendonça. 2019. Towards a Serverless Platform for Edge Computing. In *IEEE International Conference on Fog Computing (ICFC)*.

[11] Luciano Baresi, Danilo Filgueira Mendonça, and Martin Garriga. 2017. Empowering Low-Latency Applications Through a Serverless Edge Computing Architecture. In *European Conf. on Service-Oriented and Cloud Comp. (ESOCC)*.

[12] Iqra Batool and Sania Kanwal. 2025. Serverless Edge Computing: A Taxonomy, Systematic Literature Review, Current Trends and Research Challenges. *CoRR* abs/2502.15775 (2025).

[13] André Bauer, Haochen Pan, Ryan Chard, Yadu N. Babuji, Josh Bryan, Devesh Tiwari, Ian T. Foster, and Kyle Chard. 2024. The globus compute dataset: An open function-as-a-service dataset from the edge to the cloud. *Future Gener. Comput. Syst.* 153 (2024), 558–574.

[14] Zouhir Bellal, Laaziz Lahlou, Nadjia Kara, Timothy Murphy, and Tan Phat Nguyen. 2025. 3GC: A Deadline-Aware and Energy-Efficient Resource Allocation Scheme for Serverless Edge Computing. In *IEEE Consumer Communications & Networking Conference (CNCC)*.

[15] Priscilla Benedetti, Giuseppe Coviello, Kunal Rao, and Srimat Chakradhar. 2022. DataX Allocator: Dynamic resource management for stream analytics at the Edge. In *International Conference on Internet of Things: Systems, Management and Security (IOTSMS)*.

[16] Priscilla Benedetti, Mauro Femminella, and Gianluca Reali. 2026. Management of autoscaling serverless functions in edge computing via Q-Learning. *Future Gener. Comput. Syst.* 175 (2026), 108112.

[17] Priscilla Benedetti, Mauro Femminella, Gianluca Reali, and Kris Steenhaut. 2021. Experimental Analysis of the Application of Serverless Computing to IoT Platforms. *Sensors* (2021).

[18] Priscilla Benedetti, Mauro Femminella, Gianluca Reali, and Kris Steenhaut. 2022. Reinforcement Learning Applicability for Resource-Based Auto-scaling in Serverless Edge Applications. In *IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops)*.

[19] Mounir Bensalem, Francisco Carpio, and Admela Jukan. 2023. Towards Optimal Serverless Function Scaling in Edge Computing Network. In *IEEE International Conference on Communications (ICC)*.

[20] Cecilia Calavaro, Gabriele Russo Russo, Martina Salvati, Valeria Cardellini, and Francesco Lo Presti. 2024. Towards Energy-Aware Execution and Offloading of Serverless Functions. In *International Workshop on Flexible Resource and Application Management on the Edge (FRAME)*.

[21] Kun Cao and Jian Weng. 2024. REPFS: Reliability-Ensured Personalized Function Scheduling in Sustainable Serverless Edge Computing. *IEEE Transactions on Sustainable Computing* (2024).

[22] Francisco Carpio, Marc Michalke, and Admela Jukan. 2021. Engineering and Experimentally Benchmarking a Serverless Edge Computing System. In *IEEE Global Communications Conference (GLOBECOM)*.

[23] Francisco Carpio, Marc Michalke, and Admela Jukan. 2023. BenchFaaS: Benchmarking Serverless Functions in an Edge Computing Network Testbed. *IEEE Network* (2023).

[24] Gustavo André Setti Cassel, Vinicius Facco Rodrigues, Rodrigo da Rosa Righi, Marta Rosecler Bez, Andressa Cruz Nepomuceno, and Cristiano André da Costa. 2022. Serverless computing for Internet of Things: A systematic literature review. *Future Generation Computer Systems* 128 (2022), 299–316.

[25] Paul C. Castro, Vatche Ishakian, Vinod Muthusamy, and Aleksander Slominski. 2019. The rise of serverless computing. *Communications of ACM* 62, 12 (2019), 44–54.

[26] Chen Chen, Peiyuan Guan, Ziru Chen, Amir Taherkordi, Fen Hou, and Lin X. Cai. 2025. Cost Optimization for Serverless Edge Computing with Budget Constraints Using Deep Reinforcement Learning. In *IEEE International Conference on Communications (ICC)*.

[27] Chen Chen, Manuel Herrera, Ge Zheng, Liqiao Xia, Zhengyang Ling, and Jiangtao Wang. 2024. Cross-Edge Orchestration of Serverless Functions With Probabilistic Caching. *IEEE Transactions on Services Computing* (2024).

[28] Chen Chen, Lars Nagel, Lin Cui, and Fung Po Tso. 2023. S-Cache: Function Caching for Serverless Edge Computing. In *International Workshop on Edge Systems, Analytics and Networking (EdgeSys)*.

[29] Junhong Chen, Yanying Lin, Shijie Peng, Shuaipeng Wu, Kenneth B. Kent, Hao Dai, Kejiang Ye, and Yang Wang. 2025. Understanding Serverless Inference in Mobile-Edge Networks: A Benchmark Approach. *IEEE Transactions on Cloud Computing* 13 (2025).

[30] Xin Chen, Manoj Prabhakar Paidiparthy, Chen Qian, and Liting Hu. 2025. Capybara: an Edge-Friendly Distributed Object Store for Diverse Serverless Functions. In *International Middleware Conference (MIDDLEWARE)*.

[31] Xin Chen, Manoj Prabhakar Paidiparthy, Dilma Da Silva, and Liting Hu. 2025. Ekko: Fully Decentralized Scheduling for Serverless Edge Computing. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*.

[32] Bin Cheng, Jonathan Fürst, Gürkan Solmaz, and Takuya Sanada. 2019. Fog Function: Serverless Fog Computing for Data Intensive IoT Services. In *IEEE International Conference on Services Computing (SCC)*.

[33] Armin Choupani, Sadoon Azizi, and Mohammad Sadegh Aslanpour. 2025. Joint resource autoscaling and request scheduling for serverless edge computing. *Cluster Computing* 28, 3 (2025).

[34] Claudio Cicconetti, Raffaele Bruno, and Andrea Passarella. 2024. Energy-Efficient Deployment of Stateful FaaS Vertical Applications on Edge Data Networks. In *International Conf. on Computer Comm. and Networks (ICCCN)*.

[35] Claudio Cicconetti, Marco Conti, and Andrea Passarella. 2020. Architecture and performance evaluation of distributed computation offloading in edge computing. *Simulation Modelling Practice and Theory* 101 (2020), 102007.

[36] Claudio Cicconetti, Marco Conti, and Andrea Passarella. 2021. A Decentralized Framework for Serverless Edge Computing in the Internet of Things. *IEEE Transactions on Network and Service Management* (2021).

[37] Claudio Cicconetti, Marco Conti, and Andrea Passarella. 2021. On Realizing Stateful FaaS in Serverless Edge Networks: State Propagation. In *IEEE International Conference on Smart Computing (SMARTCOMP)*.

[38] Claudio Cicconetti, Marco Conti, and Andrea Passarella. 2022. FaaS execution models for edge applications. *Pervasive Mob. Comput.* 86 (2022), 101689.

[39] Claudio Cicconetti, Marco Conti, and Andrea Passarella. 2022. In-Network Computing With Function as a Service at the Edge. *Computer* 55, 9 (2022), 65–73.

[40] Claudio Cicconetti, Marco Conti, Andrea Passarella, and Dario Sabella. 2020. Toward Distributed Computing Environments with Serverless Solutions in Edge Systems. *IEEE Communications Magazine* (2020).

[41] Jaime Dantas, Hamzeh Khazaei, and Marin Litoiu. 2022. Green LAC: Resource-Aware Dynamic Load Balancer for Serverless Edge Computing Platforms. In *International Conference on Computer Science and Soft. Eng. (CASCON)*.

[42] Cailian Deng, Xuming Fang, Xianbin Wang, and Kevin Law. 2022. Software Orchestrated and Hardware Accelerated Artificial Intelligence: Toward Low Latency Edge Computing. *IEEE Wireless Communications* (2022).

[43] Shuiguang Deng, Hailiang Zhao, Zhengzhe Xiang, Cheng Zhang, Rong Jiang, Ying Li, Jianwei Yin, Schahram Dustdar, and Albert Y. Zomaya. 2022. Dependent Function Embedding for Distributed Serverless Edge Computing. *IEEE Transactions on Parallel and Distributed Systems* (2022).

[44] Dong Du, Tianyi Yu, Yiwen Zhang, Ziqian Wang, Junchen Jiang, Zhihao Jia, Xupeng Wang, Jie Wu, Yongqiang Xiong, Peng Cheng, Guanglu Sun, and Xiaobo Zhou. 2020. Catalyzer: Sub-millisecond Startup for Serverless Computing with Initialization-less Booting. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.

[45] Ana Ebrahimi, Mostafa Ghobaei-Arani, and Hadi Saboohi. 2024. Cold start latency mitigation mechanisms in serverless computing: Taxonomy, review, and future directions. *Journal of Systems Architecture* 151 (2024), 103115.

[46] Juan José López Escobar, Rebeca P. Díaz Redondo, and Felipe J. Gil-Castiñeira. 2024. Unleashing the power of decentralized serverless IoT dataflow architecture for the Cloud-to-Edge Continuum: a performance comparison. *Annales des Télécommunications* 79 (2024).

[47] Reza Farahani, Narges Mehran, Sashko Ristov, and Radu Prodan. 2024. HEFTLess: A Bi-Objective Serverless Workflow Batch Orchestration on the Computing Continuum. In *IEEE International Conference on Cluster Computing (CLUSTER)*.

[48] Mauro Femminella and Gianluca Reali. 2024. Application of Proximal Policy Optimization for Resource Orchestration in Serverless Edge Computing. *Computers* (2024).

[49] Mauro Femminella and Gianluca Reali. 2025. Edge Serverless Autoscaling managed via Proximal Policy Optimization. In *International Conference on Computer Communications and Networks (ICCCN)*.

[50] Yicheng Feng, Shihao Shen, Xiaofei Wang, Qiao Xiang, Hong Xu, Chenren Xu, and Wenyu Wang. 2024. BREAK: A Holistic Approach for Efficient Container Deployment among Edge Clouds. In *IEEE Conference on Computer Communications (INFOCOM)*.

[51] Nikos Filinis, Ioannis Tzanettis, Dimitrios Spatharakis, Eleni Fotopoulou, Ioannis Dimolitsas, Anastasios Zafeiropoulos, Constantinos Vassilakis, and Symeon Papavassiliou. 2024. Intent-driven orchestration of serverless applications in the computing continuum. *Future Gener. Comput. Syst.* 154 (2024).

[52] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. 2024. ServerlessLLM: Low-Latency Serverless Inference for Large Language Models. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

[53] Philipp Gackstatter, Pantelis A. Frangoudis, and Schahram Dustdar. 2022. Pushing Serverless to the Edge with WebAssembly Runtimes. In *IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*.

[54] Phani Kishore Gadepalli, Sean McBride, Gregor Peach, Ludmila Cherkasova, and Gabriel Parmer. 2020. Sledge: a Serverless-first, Light-weight Wasm Runtime for the Edge. In *International Middleware Conference (Middleware)*.

[55] Enrico Galimberti, Bruno Guindani, Federica Filippini, Hamta Sedghani, Danilo Ardagna, Germán Moltó, and Miguel Caballer. 2023. OSCAR-P and aMLLibrary: Performance Profiling and Prediction of Computing Continua Applications. In *ACM/SPEC International Conference on Performance Engineering (ICPE)*.

[56] Fabian Gand, Ilenia Fronza, Nabil El Ioini, Hamid R. Barzegar, Shelernaz Azimi, and Claus Pahl. 2020. Fuzzy Container Orchestration for Self-adaptive Edge Architectures. In *Int'l Conf. on Cloud Computing and Services Science (CLOSER)*.

[57] Andrea Garbugli, Andrea Sabbioni, Antonio Corradi, and Paolo Bellavista. 2022. TEMPOS: QoS Management Middleware for Edge Cloud Computing FaaS in the Internet of Things. *IEEE Access* 10 (2022), 49114–49127.

[58] Hongmin Geng, Deze Zeng, Yuepeng Li, Lin Gu, Quan Chen, and Peng Li. 2024. PLAYS: Minimizing DNN Inference Latency in Serverless Edge Cloud for Artificial Intelligence of Things. *IEEE Internet Things J.* 11, 23 (2024).

[59] Mohammad Mahdi Ghaseminya, Seyed Abolfazl Shahzadeh Fazeli, Jamshid Abouei, and Elham Abbasi. 2025. Fogfaas: providing serverless computing simulation for iFogSim and edge cloud. *Journal of Supercomputing* 81, 7 (2025).

[60] Mohsen Ghorbian, Mostafa Ghobaei-Arani, and Leila Esmaeili. 2024. A survey on the scheduling mechanisms in serverless computing: a taxonomy, challenges, and trends. *Cluster Computing* 27, 5 (2024), 5571–5610.

[61] Muhammed Golec, Guneet Walia, Mohit, Félix Cuadrado, Sukhpal Gill, and Steve Uhlig. 2025. Cold Start Latency in Serverless Computing: A Systematic Review, Taxonomy, and Future Directions. *Comput. Surveys* 57, 3 (2025).

[62] Catarina Gonçalves, José Simão, and Luís Veiga. 2026. A function-as-a-service middleware for decentralized collaborative edge computing. *Future Gener. Comput. Syst.* 175 (2026).

[63] Luis Miguel Vaquero González and Luis Rodero-Merino. 2014. Finding your Way in the Fog: Towards a Comprehensive Definition of Fog Computing. *Computer Communication Review* (2014).

[64] Huixian Gu, Liqiang Zhao, Zhu Han, Gan Zheng, and Shenghui Song. 2024. AI-Enhanced Cloud-Edge-Terminal Collaborative Network: Survey, Applications, and Future Directions. *IEEE Comm. Surveys & Tutorials* 26, 2 (2024).

[65] Peiyuan Guan, Chen Chen, Ziru Chen, Lin X. Cai, Xing Hao, and Amir Taherkordi. 2024. Context-aware Container Orchestration in Serverless Edge Computing. In *IEEE Global Communications Conference (GLOBECOM)*.

[66] Xiaolin Guo, Fang Dong, Dian Shen, Zhaowu Huang, and Jinghui Zhang. 2025. Resource-Efficient DNN Inference With Early Exiting in Serverless Edge Computing. *IEEE Transactions on Mobile Computing* 24, 5 (2025).

[67] Sabyasachi Gupta, Paul Gratz, and John Lusher. 2025. KiSS: A Novel Container Size-Aware Memory Management Policy for Serverless in Edge-Cloud Continuum. *CoRR* abs/2502.12540 (2025).

[68] Samaneh H.-Mahdizadeh-Zargar and Saeid Abrishami. 2024. An assignment mechanism for workflow scheduling in Function as a Service edge environment. *Future Gener. Comput. Syst.* 157 (2024), 543–557.

[69] Ilyas Hadjou and Young-Woo Kwon. 2024. RL-Based Approach to Enhance Reliability and Efficiency in Autoscaling for Heterogeneous Edge Serverless Computing Environments. In *IEEE Pacific Rim International Symposium on Dependable Computing (PRDC)*.

[70] Adam Hall and Umakishore Ramachandran. 2019. An execution model for serverless functions at the edge. In *ACM/IEEE International Conference on Internet of Things Design and Implementation (IoTDI)*.

[71] Adam Hall and Umakishore Ramachandran. 2025. A Hybrid Runtime for Function-as-a-Service at the Edge. In *International Middleware Conference (MIDDLEWARE)*.

[72] Jianwei Hao, Piyush Subedi, Lakshmish Ramaswamy, and In Kee Kim. 2023. Reaching for the Sky: Maximizing Deep Learning Inference Throughput on Edge Devices with AI Multi-Tenancy. *ACM Transactions on Internet Technology* 23, 1 (2023), 2:1–2:33.

[73] Hassan B. Hassan, Saman A. Barakat, and Qusay Idrees Sarhan. 2021. Survey on serverless computing. *Journal of Cloud Computing* 10, 1 (2021), 39.

[74] Adam Hasselberg, Thomas Ohlson Timoudas, Paris Carbone, and György Dán. 2024. Cliffhanger: An Experimental Evaluation of Stateful Serverless at the Edge. In *Wireless On-Demand Network Sys. and Services Conf. (WONS)*.

[75] Cheol-Ho Hong and Blesson Varghese. 2019. Resource Management in Fog/Edge Computing: A Survey on Architectures, Infrastructure, and Algorithms. *Comput. Surveys* 52, 5 (2019), 97:1–97:37.

[76] Shihong Hu, Zhihao Qu, Bin Tang, Baoliu Ye, Guanghui Li, and Weisong Shi. 2024. Joint Service Request Scheduling and Container Retention in Serverless Edge Computing for Vehicle-Infrastructure Collaboration. *IEEE Transactions on Mobile Computing* (2024).

[77] Yaodong Huang, Zelin Lin, Tingting Yao, Changkang Mo, Xiaojun Shang, Laizhong Cui, and Yuanyuan Yang. 2024. Mobility-Aware Seamless Virtual Function Migration in Deviceless Edge Computing Environments. *IEEE Transactions on Mobile Computing* (2024).

[78] Huawei Cloud. 2025. Huawei Cloud Production FaaS Trace Data. https://github.com/sir-lab/data-release.

[79] Nathaniel Hudson, Hana Khamfroush, Matt Baughman, Daniel E. Lucani, Kyle Chard, and Ian T. Foster. 2024. QoS-aware edge AI placement and scheduling with multiple implementations in FaaS-based edge computing. *Future Gener. Comput. Syst.* (2024).

[80] IBM. 2025. IBM Cloud Code Engine Traces. https://github.com/sir-lab/data-release.

[81] Sundas Iftikhar, Sukhpal Singh Gill, Chenghao Song, Minxian Xu, Mohammad Sadegh Aslanpour, Adel N. Toosi, Junhui Du, Huaming Wu, Shreya Ghosh, Deepraj Chowdhury, Muhammed Golec, Mohit Kumar, Ahmed M. Abdelmoniem, Felix Cuadrado, Blesson Varghese, Omer Rana, Schahram Dustdar, and Steve Uhlig. 2023. AI-based fog and edge computing: A systematic review, taxonomy and future directions. *Internet of Things* 21 (2023), 100674.

[82] Nabil El Ioini, David Hästbacka, Claus Pahl, and Davide Taibi. 2020. Platforms for Serverless at the Edge: A Review. In *European Conference on Service-Oriented and Cloud Computing (ESOCC)*.

[83] Prakhar Jain, Prakhar Singhal, Divyansh Pandey, Giovanni Quatrocchi, and Karthik Vaidhyanathan. 2024. POSEIDON: Efficient Function Placement at the Edge Using Deep Reinforcement Learning. In *International Conference on Service-Oriented Computing (ICSOC)*.

[84] Rutvij H. Jhaveri, Hao Ran Chi, and Huaming Wu. 2024. TinyML for Empowering Low-Power IoT Edge Consumer Devices. *IEEE Transactions on Consumer Electronics* 70, 4 (2024), 7318–7321.

[85] Fan Jia, Guojie Luo, Xiang-Yu Zhang, Hao Dai, Lei Liu, Hao Zhang, and Wei Lin. 2022. CoDL: Efficient CPU-GPU Co-Execution for Deep Learning Inference at the Edge. In *International Conference on Parallel Processing (ICPP)*.

[86] Zhiming Jia and Emmett Witchel. 2021. Boki: Stateful Serverless Computing with Shared Logs. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*.

[87] Anshul Jindal, Michael Gerndt, Mohak Chadha, Vladimir Podolskiy, and Pengfei Chen. 2021. Function delivery network: Extending serverless computing for heterogeneous platforms. *Software: Practice and Experience* (2021).

[88] João Bachiega Jr., Breno G. S. Costa, Leonardo Rebouças de Carvalho, Michel J. F. Rosa, and Aletéia P. F. Araújo. 2023. Computational Resource Allocation in Fog Computing: A Comprehensive Survey. *Comput. Surveys* 55, 14s (2023), 1–31.

[89] Pekka Karhula, Jan Janak, and Henning Schulzrinne. 2019. Checkpointing and Migration of IoT Edge Functions. In *International Workshop on Edge Systems, Analytics and Networking (EdgeSys@EuroSys)*.

[90] Kate Keahey, Michael Sherman, Jason Anderson, and Mark Powers. 2025. CHI@Edge: Supporting Experimentation in the Edge to Cloud Continuum. In *Practice and Experience in Advanced Research Computing (PEARC)*.

[91] Mina Emami Khansari and Saeed Sharifian. 2025. A deep reinforcement learning approach towards distributed Function as a Service (FaaS) based edge application orchestration in cloud-edge continuum. *Journal of Network and Computer Applications* 233 (2025).

[92] Jeongchul Kim and Kyungyong Lee. 2019. FunctionBench: A Suite of Workloads for Serverless Cloud Function Service. In *IEEE International Conference on Cloud Computing (CLOUD)*.

[93] Myung-Hyun Kim, Jaehak Lee, Heon-Chang Yu, and EunYoung Lee. 2023. Improving Memory Utilization by Sharing DNN Models for Serverless Inference. In *IEEE International Conference on Consumer Electronics (ICCE)*.

[94] Vojdan Kjorveziroski and Sonja Filiposka. 2022. Kubernetes distributions for the edge: serverless performance evaluation. *Journal of Supercomputing* (2022).

[95] Vojdan Kjorveziroski, Sonja Filiposka, and Vladimir Trajkovik. 2021. IoT Serverless Computing at the Edge: A Systematic Mapping Review. *Computers* 10, 10 (2021), 130.

[96] Haneul Ko, Hyeonjae Jeong, Daeyoung Jung, and Sangheon Pack. 2024. Dynamic Split Computing Framework in Distributed Serverless Edge Clouds. *IEEE Internet Things J.* (2024).

[97] Haneul Ko and Sangheon Pack. 2023. Function-Aware Resource Management Framework for Serverless Edge Computing. *IEEE Internet Things J.* 10, 2 (2023), 1310–1319.

[98] Dechao Kong, Yao Tan, S. M. Noman Islam, Ruiying Huang, Chen Chen, Zhao Wang, Long Jin, Peng Zeng, Muhammad Asghar Khan, and Sanjay Kumar Das. 2022. Edge-computing-driven Internet of Things: A Survey. *Comput. Surveys* 55, 8 (2022), 1–41.

[99] Sameer G. Kulkarni, Guyue Liu, K. K. Ramakrishnan, and Timothy Wood. 2019. Living on the Edge: Serverless Computing and the Cost of Failure Resiliency. In *IEEE Int'l Symp. on Local and Metropolitan Area Net. (LANMAN)*.

[100] M. Rudra Kumar, B. Rupa Devi, K. Rangaswamy, M. Sangeetha, and Korupalli V. Rajesh Kumar. 2023. IoT-Edge Computing for Efficient and Effective Information Process on Industrial Automation. In *International Conference on Networking and Communications (ICNWC)*.

[101] Wonhee Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *ACM Symposium on Operating Systems Principles (SOSP)*.

[102] Vincent Lannurien, Camélia Slimani, Laurent d'Orazio, Olivier Barais, Stéphane Paquelet, and Jalil Boukhobza. 2024. HeROcache: Storage-Aware Scheduling in Heterogeneous Serverless Edge - The Case of IDS. In *IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*.

[103] Cherif Latreche, Nikos Parlavantzas, and Hector A. Duran-Limon. 2024. FoRLess: A Deep Reinforcement Learning-based approach for FaaS Placement in Fog. In *IEEE/ACM International Conf. on Utility and Cloud Comp. (UCC)*.

[104] Nikita Lazarev, Varun Gohil, James Tsai, Andy Anderson, Bhushan Chitlur, Zhiru Zhang, and Christina Delimitrou. 2024. Sabre: Hardware-Accelerated Snapshot Compression for Serverless MicroVMs. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

[105] Jaden Jinu Lee, Judah Nava, and Hanku Lee. 2024. ComFaaS Distributed: Edge Computing with Function-as-a-Service in Parallel Cloud Environments. In *International Conference on Information and Computer Technologies (ICICT)*.

[106] Kexin Li and Stefan Nastic. 2023. AttentionFunc: Balancing FaaS Compute across Edge-Cloud Continuum with Reinforcement Learning. In *International Conference on the Internet of Things (IoT)*.

[107] Ming Li, Furong Xu, Yuqin Wu, Jianshan Zhang, Weitao Xu, and Yuezhong Wu. 2025. Real-time task dispatching and scheduling in serverless edge computing. *Ad Hoc Networks* 174 (2025).

[108] Ming Li, Jianshan Zhang, Jingfeng Lin, Zheyi Chen, and Xianghan Zheng. 2023. FireFace: Leveraging Internal Function Features for Configuration of Functions on Serverless Edge Platforms. *Sensors* (2023).

[109] Xin Li, Long Chen, Zian Yuan, and Guangrui Liu. 2025. AIHO: Enhancing task offloading and reducing latency in serverless multi-edge-to-cloud systems. *Future Gener. Comput. Syst.* 165 (2025).

[110] Xue Li, Peng Kang, Jordan Molone, Wei Wang, and Palden Lama. 2022. KneeScale: Efficient Resource Scaling for Serverless Computing at the Edge. In *IEEE International Symp. on Cluster, Cloud and Internet Comp. (CCGrid)*.

[111] Yongkang Li, Yanying Lin, Yang Wang, Kejiang Ye, and Cheng-Zhong Xu. 2023. Serverless Computing: State-of-the-Art, Challenges and Opportunities. *IEEE Transactions on Services Computing* 16, 2 (2023), 1522–1539.

[112] Yunqi Li and Changlin Yang. 2024. Resource Allocation and Pricing in Energy Harvesting Serverless Computing Internet of Things Networks. *Information* (2024).

[113] Yuepeng Li, Deze Zeng, Lin Gu, Kun Wang, and Song Guo. 2022. On the Joint Optimization of Function Assignment and Communication Scheduling toward Performance Efficient Serverless Edge Computing. In *IEEE/ACM International Symposium on Quality of Service (IWQoS)*.

[114] Zhuozhao Li, Ryan Chard, Yadu N. Babuji, Ben Galewsky, Tyler J. Skluzacek, Kirill Nagaitsev, Anna Woodard, Ben Blaiszik, Josh Bryan, Daniel S. Katz, Ian T. Foster, and Kyle Chard. 2022. funcX: Federated Function as a Service for Science. *IEEE Transactions on Parallel and Distributed Systems* (2022).

[115] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: Efficient Serving of LLM-based Applications with Semantic Variable. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

[116] Wentao Liu, Ruiting Zhou, and Yue Ma. 2025. Sdser: Online Deployment and Scheduling of Dynamic DAG Functions with Bayesian Prediction in Serverless Edge Computing. In *IEEE International Conf. on Distributed Comp. Sys. (ICDCS)*.

[117] Davide Loconte, Saverio Ieva, Filippo Gramegna, Ivano Bilenchi, Corrado Fasciano, and Agnese Pinto. 2024. Serverless microservice architecture for cloud-edge intelligence in sensor networks. *IEEE Sensors Journal* (2024).

[118] Jiong Lou, Zhiqing Tang, Xinyu Lu, Shijing Yuan, Jie Li, Weijia Jia, and Chentao Wu. 2024. Efficient Serverless Function Scheduling in Edge Computing. In *IEEE International Conference on Communications (ICC)*.

[119] Zhenyan Lu, Xiang Li, Dongqi Cai, Rongjie Yi, Fangming Liu, Wei Liu, Jian Luan, Xiwen Zhang, Nicholas D. Lane, and Mengwei Xu. 2025. Demystifying Small Language Models for Edge Deployment. In *Annual Meeting of the Association for Computational Linguistics (ACL)*.

[120] Ke Luo, Tao Ouyang, Zhi Zhou, and Xu Chen. 2023. BeeFlow: Behavior tree-based Serverless workflow modeling and scheduling for resource-constrained edge clusters. *Journal of Systems Architecture* (2023).

[121] Quyuan Luo, Shihong Hu, Changle Li, Guanghui Li, and Weisong Shi. 2021. Resource Scheduling in Edge Computing: A Survey. *IEEE Communications Surveys & Tutorials* 23, 4 (2021), 2131–2165.

[122] Xiaosu Lyu, Emil Abbasov, Sean McBride, Gabriel Parmer, and Timothy Wood. 2025. SledgeScale: Load-Aware Dispatch and Deadline-Driven Scheduling for Scalable, Dense Serverless Computing in Edge Data Centers. In *ACM/IEEE Conference on Edge Systems (SEC)*.

[123] Md. Redowan Mahmud, Samodha Pallewatta, Mohammad Goudarzi, and Rajkumar Buyya. 2022. iFogSim2: An extended iFogSim simulator for mobility, clustering, and microservice management in edge and fog computing environments. *J. Syst. Softw.* 190 (2022), 111351.

[124] Vincenzo De Maio, David Bermbach, and Ivona Brandic. 2022. TAROT: Spatio-Temporal Function Placement for Serverless Smart City Applications. In *IEEE/ACM International Conference on Utility and Cloud Computing (UCC)*.

[125] Amjad Yousef Majid and Eduard Marin. 2023. A Review of Deep Reinforcement Learning in Serverless Computing: Function Scheduling and Resource Auto-Scaling. *CoRR* abs/2311.12839 (2023).

[126] Mohammadreza Malekabbasi, Tobias Pfandzelter, and David Bermbach. 2025. Umbilical Choir: Automated Live Testing for Edge-to-Cloud FaaS Applications. In *IEEE International Conference on Fog and Edge Computing (ICFEC)*.

[127] Anupama Mampage, Shanika Karunasekera, and Rajkumar Buyya. 2022. A Holistic View on Resource Management in Serverless Computing Environments: Taxonomy and Future Directions. *Comput. Surveys* 54, 11s (2022), 222:1–222:36.

[128] Angelo Marchese and Orazio Tomarchio. 2023. Orchestrating serverless applications in the Cloud-to-Edge continuum. In *International Workshop on Middleware for the Computing Continuum (Mid4CC)*.

[129] MarketsandMarkets. 2024. Serverless Architecture Market by Service Model (BaaS, FaaS), Deployment Mode, Organization Size, Vertical - Global Forecast to 2029. https://www.marketsandmarkets.com/Market-Reports/serverless-computing-market-217021547.html.

[130] Microsoft Azure. 2019. Azure Functions Dataset 2019. https://github.com/Azure/AzurePublicDataset/blob/master/AzureFunctionsDataset2019.md.

[131] Adriana Mijuskovic, Alessandro Chiumento, Rob H. Bemthuis, Adina Aldea, and Paul J. M. Havinga. 2021. Resource Management Techniques for Cloud/Fog and Edge Computing: An Evaluation Framework and Classification. *Sensors* 21, 5 (2021), 1832.

[132] Chetankumar Mistry, Bogdan Stelea, Vijay Kumar, and Thomas F. J.-M. Pasquier. 2020. Demonstrating the Practicality of Unikernels to Build a Serverless Platform at the Edge. In *IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*.

[133] Viyom Mittal, Shixiong Qi, Ratnadeep Bhattacharya, Xiaosu Lyu, Junfeng Li, Sameer G. Kulkarni, Dan Li, Jinho Hwang, K. K. Ramakrishnan, and Timothy Wood. 2021. Mu: An Efficient, Fair and Responsive Serverless Framework for Resource-Constrained Edge Clouds. In *ACM Symposium on Cloud Computing (SoCC)*.

[134] Felix Moebius, Tobias Pfandzelter, and David Bermbach. 2024. Are Unikernels Ready for Serverless on the Edge?. In *IEEE International Conference on Cloud Engineering (IC2E)*.

[135] Stefan Nastic, Thomas Rausch, Ognjen Scekic, Schahram Dustdar, Marjan Gusev, Bojana Koteska, Magdalena Kostoska, Boro Jakimovski, Sasko Ristov, and Radu Prodan. 2017. A Serverless Real-Time Data Analytics Platform for Edge Computing. *IEEE Internet Computing* (2017).

[136] Hai Duc Nguyen and Andrew A. Chien. 2023. Storm-RTS: Stream Processing with Stable Performance for Multi-Cloud and Cloud-edge. In *IEEE International Conference on Cloud Computing (CLOUD)*.

[137] Andrei Palade, Atri Mukhopadhyay, Aqeel H. Kazmi, Christian Cabrera, Evelyn Nomayo, Georgios Iosifidis, Marco Ruffini, and Siobhán Clarke. 2020. A Swarm-based Approach for Function Placement in Federated Edges. In *IEEE International Conference on Services Computing (SCC)*.

[138] Giuseppe De Palma, Saverio Giallorenzo, Jacopo Mauro, Matteo Trentin, and Gianluigi Zavattaro. 2022. A Declarative Approach to Topology-Aware Serverless Function-Execution Scheduling. In *IEEE International Conference on Web Services (ICWS)*.

[139] Giuseppe De Palma, Saverio Giallorenzo, Jacopo Mauro, Matteo Trentin, and Gianluigi Zavattaro. 2024. WebAssembly at the Edge: Benchmarking a Serverless Platform for Private Edge Cloud Systems. *IEEE Internet Computing* 28 (2024).

[140] Li Pan, Lin Wang, Shutong Chen, and Fangming Liu. 2022. Retention-Aware Container Caching for Serverless Edge Computing. In *IEEE Conference on Computer Communications (INFOCOM)*.

[141] Manish Pandey, Byungchul Tak, and Young-Woo Kwon. 2025. PROBA: Enhancing Serverless Edge Computing via Adaptive Task Scheduling and Probabilistic Resource Sharing. In *IEEE International Conf. on Cloud Comp. (CLOUD)*.

[142] Yashwant Singh Patel and Paul Townend. 2024. A Stable Matching Approach to Energy Efficient and Sustainable Serverless Scheduling for the Green Cloud Continuum. In *IEEE International Conference on Service-Oriented System Engineering (SOSE)*.

[143] Liam Patterson, David Pigorovsky, Brian Dempsey, Nikita Lazarev, Aditya Shah, Clara Steinhoff, Ariana Bruno, Justin Hu, and Christina Delimitrou. 2022. HiveMind: a hardware-software system stack for serverless edge swarms. In *Annual International Symposium on Computer Architecture (ISCA)*.

[144] István Pelle, Francesco Paolucci, Balázs Sonkoly, and Filippo Cugini. 2021. Latency-Sensitive Edge/Cloud Serverless Dynamic Deployment Over Telemetry-Based Packet-Optical Network. *IEEE Journal on Sel. Areas in Comm.* (2021).

[145] Haosong Peng, Yufeng Zhan, Peng Li, and Yuanqing Xia. 2024. Tangram: High-Resolution Video Analytics on Serverless Platform with SLO-Aware Batching. In *IEEE International Conf. on Distributed Comp. Sys. (ICDCS)*.

[146] Per Persson and Ola Angelsmark. 2017. Kappa: serverless IoT deployment. In *International Workshop on Serverless Computing (WOSC@Middleware)*.

[147] Emanuele Petriglia, Federica Filippini, Giacomo Pracucci, Marco Savi, and Michele Ciavotta. 2024. Comparing actor-critic and neuroevolution approaches for traffic offloading in FaaS-powered edge systems. In *1st Workshop on Serverless at the Edge (SEATED)*.

[148] Tobias Pfandzelter and David Bermbach. 2020. tinyFaaS: A Lightweight FaaS Platform for Edge Environments. In *IEEE International Conference on Fog Computing (ICFC)*.

[149] Olga Poppe, Qun Guo, Willis Lang, Pankaj Arora, Morgan Oslake, Shize Xu, and Ajay Kalhan. 2022. Moneyball: Proactive Auto-Scaling in Microsoft Azure SQL Database Serverless. *Proc. VLDB Endow.* (2022).

[150] Mega Pranata, Aditya Wijayanto, and Muhammad Fajar Sidiq. 2023. Serverless Autoscaling Metrics for Optimum Performance on Edge Computing. In *International Sem. on Research of Info. Tech. and Intelligent Sys. (ISRITI)*.

[151] Carlo Puliafito, Claudio Cicconetti, Marco Conti, Enzo Mingozzi, and Andrea Passarella. 2023. Balancing local vs. remote state allocation for micro-services in the cloud-edge continuum. *Pervasive and Mobile Computing* (2023).

[152] Thomas W. Pusztai, Cynthia Marcelino, and Stefan Nastic. 2024. HyperDrive: Scheduling Serverless Functions in the Edge-Cloud-Space 3D Continuum. In *IEEE/ACM Symposium on Edge Computing (SEC)*.

[153] Sheshadri K. R and J. Lakshmi. 2024. RightFusion: Enabling QoS driven Function Fusion in Edge-Cloud FaaS. In *IEEE International Conference on Cloud Engineering (IC2E)*.

[154] Ana Radovanović, Ross Koningstein, Ian Schneider, Bokan Chen, Alexandre Duarte, Binz Roy, Diyue Xiao, Maya Haridasan, Patrick Hung, Nick Care, Saurav Talukdar, Eric Mullen, Kendal Smith, MariEllen Cottman, and Walfredo Cirne. 2023. Carbon-Aware Computing for Datacenters. *IEEE Transactions on Power Systems* 38, 2 (2023), 1270–1280.

[155] Philipp Raith, Stefan Nastic, and Schahram Dustdar. 2023. Serverless Edge Computing - Where We Are and What Lies Ahead. *IEEE Internet Computing* 27, 3 (2023), 50–64.

[156] Philipp Raith, Stefan Nastic, and Schahram Dustdar. 2024. SimuScale: Optimizing Parameters for Autoscaling of Serverless Edge Functions Through Co-Simulation. In *IEEE International Conference on Cloud Computing (CLOUD)*.

[157] Philipp Raith, Thomas Rausch, Alireza Furutanpey, and Schahram Dustdar. 2023. *faas-sim*: A trace-driven simulation framework for serverless edge computing platforms. *Software: Practice and Experience* (2023).

[158] Philipp Raith, Thomas Rausch, Paul Prüller, Alireza Furutanpey, and Schahram Dustdar. 2022. An End-to-End Framework for Benchmarking Edge-Cloud Cluster Management Techniques. In *IEEE International Conference on Cloud Engineering (IC2E)*.

[159] Kaustubh Rajendra Rajput, Chinmay Dilip Kulkarni, Byungjin Cho, Wei Wang, and In Kee Kim. 2022. EdgeFaaSBench: Benchmarking Edge Devices Using Serverless Computing. In *IEEE International Conference on Edge Computing and Communications (EDGE)*.

[160] Thomas Rausch, Alexander Rashed, and Schahram Dustdar. 2021. Optimized container scheduling for data-intensive serverless edge computing. *Future Gener. Comput. Syst.* 114 (2021), 259–271.

[161] Sebastián Risco, Caterina Alarcón, Sergio Langarita, Miguel Caballer, and Germán Moltó. 2024. Rescheduling serverless workloads across the cloud-to-edge continuum. *Future Gener. Comput. Syst.* 153 (2024).

[162] Gabriele Russo Russo, Daniele Ferrarelli, Diana Pasquali, Valeria Cardellini, and Francesco Lo Presti. 2024. QoS-aware offloading policies for serverless functions in the Cloud-to-Edge continuum. *Future Gener. Comput. Syst.* 156 (2024).

[163] Gabriele Russo Russo, Tiziana Mannucci, Valeria Cardellini, and Francesco Lo Presti. 2023. Serverledge: Decentralized Function-as-a-Service for the Edge-Cloud Continuum. In *IEEE International Conference on Pervasive Computing and Communications (PerCom)*.

[164] Andrea Sabbioni, Andrea Garbugli, Luca Foschini, Antonio Corradi, and Paolo Bellavista. 2024. Serverless Computing for QoS-Effective NFV in the Cloud Edge. *IEEE Communications Magazine* (2024).

[165] Christos Sad, Dimosthenis Masouros, and Kostas Siozios. 2025. FEARLESS: A Federated Reinforcement Learning Orchestrator for Serverless Edge Swarms. *IEEE Embedded Systems Letters* 17, 1 (2025).

[166] Kengo Sasaki, Naoya Suzuki, Satoshi Makido, and Akihiro Nakao. 2016. Vehicle control system coordinated between cloud and mobile edge computing. In *Annual Conf. of the Soc. of Instrument and Control Engineers of Japan (SICE)*.

[167] Mahadev Satyanarayanan. 2017. The Emergence of Edge Computing. *Computer* 50, 1 (2017), 30–39.

[168] Mahadev Satyanarayanan, Paramvir Bahl, Ramón Cáceres, and Nigel Davies. 2009. The Case for VM-Based Cloudlets in Mobile Computing. *IEEE Pervasive Computing* 8, 4 (2009), 14–23.

[169] Johann Schleier-Smith, Vikram Sreekanti, Anurag Khandelwal, João Carreira, Neeraja Jayant Yadwadkar, Raluca Ada Popa, Joseph E. Gonzalez, Ion Stoica, and David A. Patterson. 2021. What serverless computing is and should become: the next phase of cloud computing. *Communications ACM* 64, 5 (2021), 76–84.

[170] Michael Schneider, Jason R. Rambach, and Didier Stricker. 2017. Augmented reality based on edge computing using the example of remote live support. In *IEEE International Conference on Industrial Technology (ICIT)*.

[171] Meenakshi Sethunath and Yang Peng. 2022. A joint function warm-up and request routing scheme for performing confident serverless computing. *High-Confidence Computing* 2, 3 (2022), 100071.

[172] Hossein Shafiei, Ahmad Khonsari, and Payam Mousavi. 2022. Serverless Computing: A Survey of Opportunities, Challenges, and Applications. *Comput. Surveys* 54, 11s (2022), 239:1–239:32.

[173] Mohammad Shahrad, Rodrigo Fonseca, Iñigo Goiri, Gohar Irfan Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. 2020. Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider. In *USENIX Annual Tech. Conf. (USENIX ATC)*.

[174] Xiaojun Shang, Yingling Mao, Yu Liu, Yaodong Huang, Zhenhua Liu, and Yuanyuan Yang. 2023. Online Container Scheduling for Data-intensive Applications in Serverless Edge Computing. In *IEEE Conference on Computer Communications (INFOCOM)*.

[175] Weisong Shi, Jie Cao, Quan Zhang, Youhuizi Li, and Lanyu Xu. 2016. Edge Computing: Vision and Challenges. *IEEE Internet Things J.* 3, 5 (2016), 637–646.

[176] Emilian Simion, Yuandou Wang, Hsiang-Ling Tai, Uraz Odyurt, and Zhiming Zhao. 2023. Towards Seamless Serverless Computing Across an Edge-Cloud Continuum. In *IEEE/ACM Int'l Conference on Utility and Cloud Computing (UCC)*.

[177] Fedor Smirnov, Chris Engelhardt, Jakob Mittelberger, Behnaz Pourmohseni, and Thomas Fahringer. 2021. Apollo: towards an efficient distributed orchestration of serverless function compositions in the cloud-edge continuum. In *IEEE/ACM International Conference on Utility and Cloud Computing (UCC)*.

[178] Yonglak Son, Udit Gupta, Andrew McCrabb, Young Geun Kim, Valeria Bertacco, and David Brooks. 2025. GreenScale: Carbon Optimization for Edge Computing. *IEEE Internet Things J.* 12, 16 (2025).

[179] Vikram Sreekanti, Chenggang Wu, Xiayue Charles Lin, Johann Schleier-Smith, Jose M. Faleiro, Joseph E. Gonzalez, Joseph M. Hellerstein, and Alexey Tumanov. 2020. Cloudburst: Stateful Functions-as-a-Service. *Proc. VLDB Endow.* 13, 11 (2020), 2438–2452.

[180] Shuomiao Su, Zhi Zhou, Tao Ouyang, Ruiting Zhou, and Xu Chen. 2023. Learning to Be Green: Carbon-Aware Online Control for Edge Intelligence with Colocated Learning and Inference. In *IEEE International Conference on Distributed Computing Systems (ICDCS)*.

[181] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: Dynamic Scheduling for Large Language Model Serving. In *USENIX Symposium on Operating Sys. Design and Impl. (OSDI)*.

[182] Márk Szalay, Péter Mátray, and László Toka. 2023. Real-Time FaaS: Towards a Latency Bounded Serverless Cloud. *IEEE Transactions on Cloud Computing* (2023).

[183] Qinqin Tang, Renchao Xie, Fei Richard Yu, Tianjiao Chen, Ran Zhang, Tao Huang, and Yun-Jie Liu. 2022. Distributed Task Scheduling in Serverless Edge Computing Networks for the Internet of Things: A Learning Approach. *IEEE Internet Things J.* (2022).

[184] Mohammad Tari, Mostafa Ghobaei-Arani, Jafar Pouramini, and Mohsen Ghorbian. 2024. Auto-scaling mechanisms in serverless computing: A comprehensive review. *Computer Science Review* 53 (2024), 100650.

[185] Klervie Toczé and Simin Nadjm-Tehrani. 2018. A Taxonomy for Management and Optimization of Multiple Resources in Edge Computing. *Wireless Communications and Mobile Computing* 2018 (2018), 7476201:1–7476201:23.

[186] Minh-Ngoc Tran and Young-Han Kim. 2024. Concurrent service auto-scaling for Knative resource quota-based serverless system. *Future Gener. Comput. Syst.* (2024).

[187] Minh-Ngoc Tran and Young-Han Kim. 2024. Optimized resource usage with hybrid auto-scaling system for knative serverless edge computing. *Future Gener. Comput. Syst.* 152 (2024).

[188] Feridun Tütüncüoglu, Sladana Josilo, and György Dán. 2023. Online Learning for Rate-Adaptive Task Offloading Under Latency Constraints in Serverless Edge Computing. *IEEE/ACM Transactions on Networking* (2023).

[189] Shahrokh Vahabi, Francesca Righetti, Carlo Vallati, and Nicola Tonellotto. 2025. The Impact of Prediction Models on Energy-Aware Resource Management in FaaS Platforms. *IEEE Access* 13 (2025).

[190] Aastik Verma, Anurag Satpathy, Sajal K. Das, and Sourav Kanti Addya. 2024. LEASE: Leveraging Energy-Awareness in Serverless Edge for Latency-Sensitive IoT Services. In *IEEE International Conference on Pervasive Computing and Communications Workshops and other Affiliated Events (PerCom Workshops)*.

[191] Guneet Kaur Walia, Mohit Kumar, and Sukhpal Singh Gill. 2024. AI-Empowered Fog/Edge Resource Management for IoT Applications: A Comprehensive Review, Research Challenges, and Future Perspectives. *IEEE Communications Surveys & Tutorials* 26, 1 (2024), 619–669.

[192] Bin Wang, Ahmed Ali-Eldin, and Prashant J. Shenoy. 2021. LaSS: Running Latency Sensitive Serverless Computations at the Edge. In *ACM International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*.

[193] Ian-Chin Wang, Shixiong Qi, Elizabeth Liri, and K. K. Ramakrishnan. 2021. Towards a Proactive Lightweight Serverless Edge Cloud for Internet-of-Things Applications. In *IEEE International Conference on Net., Arch. and Storage (NAS)*.

[194] Jinfeng Wen, Zhenpeng Chen, Xin Jin, and Xuanzhe Liu. 2023. Rise of the Planet of Serverless Computing: A Systematic Review. *ACM Transactions on Software Engineering and Methodology* 32, 5 (2023), 131:1–131:61.

[195] Yanyan Wen, Guangping Xu, Jianshe Wang, and Wei Hao. 2024. Low-Latency State Management for Real-Time Tasks in Edge Serverless. In *IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA)*.

[196] Rich Wolski, Chandra Krintz, Fatih Bakir, Gareth George, and Wei-Tsung Lin. 2019. CSPOT: portable, multi-scale functions-as-a-service for IoT. In *ACM/IEEE Symposium on Edge Computing (SEC)*.

[197] Li Wu, Walid A. Hanafy, Abel Souza, Khai Nguyen, Jan Harkes, David Irwin, Mahadev Satyanarayanan, and Prashant Shenoy. 2025. CarbonEdge: Leveraging Mesoscale Spatial Carbon-Intensity Variations for Low Carbon Edge Computing. In *ACM International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*.

[198] Shichao Xia, Zhixiu Yao, Yun Li, Zhitong Xing, and Shiwen Mao. 2024. Distributed Computing and Networking Coordination for Task Offloading Under Uncertainties. *IEEE Transactions on Mobile Computing* (2024).

[199] Yang Xia, Haixia Zhang, Xiaotian Zhou, and Dongfeng Yuan. 2023. Location-Aware and Delay-Minimizing Task Offloading in Vehicular Edge Computing Networks. *IEEE Transactions on Vehicular Technology* (2023).

[200] Ke Xiao, Song Yang, Fan Li, Liehuang Zhu, Xu Chen, and Xiaoming Fu. 2024. Making Serverless Not So Cold in Edge Clouds: A Cost-Effective Online Approach. *IEEE Transactions on Mobile Computing* 23 (2024).

[201] Renchao Xie, Dier Gu, Qinqin Tang, Tao Huang, and F. Richard Yu. 2022. Workflow Scheduling Using Hybrid PSO-GA Algorithm in Serverless Edge Computing for the Internet of Things. In *IEEE Vehicular Technology Conf. (VTC Spring)*.

[202] Renchao Xie, Dier Gu, Qinqin Tang, Tao Huang, and Fei Richard Yu. 2023. Workflow Scheduling in Serverless Edge Computing for the Industrial Internet of Things: A Learning Approach. *IEEE Tran on Industrial Informatics* (2023).

[203] Renchao Xie, Qinqin Tang, Shi Qiao, Han Zhu, F. Richard Yu, and Tao Huang. 2021. When Serverless Computing Meets Edge Computing: Architecture, Challenges, and Open Issues. *IEEE Wireless Comm.* 28, 5 (2021), 126–133.

[204] Zichuan Xu, Yuexin Fu, Qiufen Xia, and Hao Li. 2023. Enabling Age-Aware Big Data Analytics in Serverless Edge Clouds. In *IEEE Conference on Computer Communications (INFOCOM)*.

[205] Qirui Yang, Runyu Jin, Nabil Gandhi, Xiongzi Ge, Hoda Aghaei Khouzani, and Ming Zhao. 2024. EdgeBench: A Workflow-Based Benchmark for Edge Computing. In *Future of Information and Communication Conference (FICC)*.

[206] Yaning Yang, Xiao Du, Yutong Ye, Jiepin Ding, Ting Wang, Mingsong Chen, and Keqin Li. 2025. Multi-Objective Deep Reinforcement Learning for Function Offloading in Serverless Edge Computing. *IEEE Transactions on Services Computing* 18, 1 (2025).

[207] Xuyi Yao, Ningjiang Chen, Xuemei Yuan, and Pingjie Ou. 2023. Performance optimization of serverless edge computing function offloading based on deep reinforcement learning. *Future Generation Computer Systems* (2023).

[208] Abolfazl Younesi, Abbas Shabrang Maryan, Elyas Oustad, Zahra Najafabadi Samani, Mohsen Ansari, and Thomas Fahringer. 2025. Splitwise: Collaborative Edge–Cloud Inference for LLMs via Lyapunov-Assisted DRL. In *ACM/IEEE International Conference on Utility and Cloud Computing (UCC)*.

[209] Ethan G. Young, Pengfei Zhu, Tyler Caraza-Harter, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2019. The True Cost of Containing: A gVisor Case Study. In *USENIX Workshop on Hot Topics in Cloud Computing (HotCloud)*.

[210] Guangba Yu, Pengfei Chen, Zibin Zheng, Jingrun Zhang, Xiaoyun Li, and Zilong He. 2023. FaaSDeliver: Cost-Efficient and QoS-Aware Function Delivery in Computing Continuum. *IEEE Transactions on Services Computing* (2023).

[211] Atiya Zahed, Mostafa Ghobaei-Arani, and Leila Esmaeili. 2025. An efficient function placement approach in serverless edge computing. *Computing* 107, 3 (2025).

[212] Yue Zeng, Junlong Zhou, Baoliu Ye, Zhihao Qu, Song Guo, Tianjian Gong, and Pan Li. 2025. ExpertDRL: Request Dispatching and Instance Configuration for Serverless Edge Inference With Foundation Models. *IEEE Transactions on Mobile Computing* 24, 9 (2025).

[213] Hang Zhang, Jinsong Wang, Hongwei Zhang, and Chao Bu. 2024. Security computing resource allocation based on deep reinforcement learning in serverless multi-cloud edge computing. *Future Gener. Comput. Syst.* 151 (2024).

[214] Jun Zhang and Khaled Ben Letaief. 2020. Mobile Edge Intelligence and Computing for the Internet of Vehicles. *Proc. IEEE* (2020).

[215] Lu Zhang, Weiqi Feng, Chao Li, Xiaofeng Hou, Pengyu Wang, Jing Wang, and Minyi Guo. 2022. Tapping into NFV Environment for Opportunistic Serverless Edge Function Deployment. *IEEE Trans. Computers* 71, 10 (2022), 2698–2704.

[216] Xuan Zhang, Hongjun Gu, Guopeng Li, Xin He, and Haisheng Tan. 2023. Online Function Caching in Serverless Edge Computing. In *IEEE International Conference on Parallel and Distributed Systems (ICPADS)*.

[217] Senjiong Zheng, Bo Liu, Weiwei Lin, Xiaoying Ye, and Keqin Li. 2023. A package-aware scheduling strategy for edge serverless functions based on multi-stage optimization. *Future Generation Computer Systems* (2023).

[218] Shaoshu Zhu, Neil Hurley, and Hadi Tabatabaee Malazi. 2025. Does Energy-efficiency Mean Carbon-awareness? A Critical Evaluation of Sustainable Serverless Edge Scheduling. In *IEEE/ACM International Conference on Utility and Cloud Computing (UCC)*.